

Streaming Communication for Scalable Policy-Aware Data Exchange Systems

A DYNAMOS Case Study

Ruben Stoop

Streaming Communication for Scalable Policy-Aware Data Exchange Systems

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

by

Ruben Stoop

under the supervision of
Dr. Ana Oprescu (CCI, UvA)

July 2026



Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands



Complex Cyber-Infrastructure
(CCI)
University of Amsterdam
Amsterdam, The Netherlands

Thesis Committee

Examiner (chair): Dr. Ana Oprescu (CCI, UvA)
Second Reviewer: To be confirmed ()
Daily Supervisor: Alexandros Koufakis (CCI, UvA)

Abstract

Policy-aware data exchange systems increasingly need to move large or continuously produced datasets across independently governed services. Traditional request-response communication is simple to reason about, but it often forces complete result materialisation before downstream processing can begin. Streaming communication can reduce this bottleneck by transferring data incrementally, yet it also changes failure behaviour, buffering, backpressure, and the relationship between the data path and policy-controlled orchestration.

This thesis studies how streaming communication can be introduced into dynamically composed, policy-aware microservice systems without weakening their control model. DYNAMOS, a middleware for policy-driven data exchange developed at the University of Amsterdam, is used as the case-study system because it combines generated service chains, data-sharing archetypes, sidecar-mediated communication, and explicit policy approval. The research first compares candidate transport mechanisms in a controlled Kubernetes pipeline, using throughput, latency, pacing, correctness, recovery, and resource pressure as evaluation dimensions. It then implements three chunk-aware DYNAMOS data paths: chunked unary transfer, intra-job gRPC streaming, and RabbitMQ Streams at the agent boundary.

A local validation on bulk `dataThroughTtp` retrieval shows that all three chunk-aware implementations reduce completion time and peak memory compared with the original classic-unary baseline: at 250,000 returned rows, completion time drops by a factor of 2.1–2.5 (29.1 s to 11.7–14.1 s) and peak sampled memory drops from 3.7 GiB to 1.4–1.7 GiB, with additional checks on `computeToData` and multi-provider `dataThroughTtp`. The results show that the main improvement comes from preserving chunks through the data path rather than from streaming as a label. Direct gRPC transfer is strongest when service lifetimes are naturally coupled; brokered streams remain valuable when decoupling, retention, and replay matter. The broader conclusion is that streaming should be treated as a post-policy data-path change: the approved execution path remains policy-governed, while large result transfer becomes incremental and more scalable.

Contents

Abstract	v
Contents	vii
List of Figures	x
List of Tables	xi
Acknowledgements	xiii
1 Introduction	1
1.1 Context	1
1.2 Case-Study System	1
1.3 Problem Statement	2
1.4 Research Questions	2
1.5 Approach and Contributions	3
1.6 Outline	3
2 Background	5
2.1 Policy-Aware Data Exchange Systems	5
2.2 DYNAMOS as Case Study	5
2.3 Candidate Communication Mechanisms	6
2.3.1 gRPC streaming	6
2.3.2 Apache Kafka	6
2.3.3 RabbitMQ and RabbitMQ Streams	7
2.3.4 NATS with JetStream persistence	7
2.4 Concept Matrix	7
3 Related Work	9
3.1 Streaming communication technologies and interaction models	9
3.2 Architectural integration of streaming in adaptive microservice middleware	10
3.3 Policy enforcement and compliance under continuous data exchange	11
3.4 Resilience, backpressure, and fault handling in streaming workflows	12
3.5 Validation practices, scalability, and performance evaluation . .	12

3.6	Research Gap	13
4	Methods	15
4.1	Research Design	15
4.2	Transport Benchmark Design	15
4.2.1	Methodological positioning and scope	15
4.2.2	Common testbed	16
4.2.3	Fairness considerations	16
4.2.4	Evaluation dimensions	17
4.2.5	Scenario set and scenario-to-property mapping	17
4.2.6	Experimental factors	17
4.2.7	Result presentation and appendix boundary	19
4.3	DYNAMOS Benchmark Method	19
4.4	Validation and Reproducibility	20
4.5	Experimental Environment	20
4.6	Ethical Considerations	20
4.6.1	Ethical safeguards and compliance	21
4.6.2	Consultation and approval	22
5	Transport Benchmark	23
5.1	Results	23
5.1.1	Metric definitions	23
5.1.2	Comparison scope	25
5.1.3	Clean bulk-transfer matrix	26
5.1.4	Continuous steady-rate scenario	26
5.1.5	Backpressure scenario	27
5.1.6	Recovery scenario	28
5.1.7	Relation to prior benchmarks	29
5.1.8	Implications for the DYNAMOS integration	29
6	DYNAMOS Integration	31
6.1	Motivation	31
6.2	Execution Path	31
6.3	Common Chunk Model	32
6.4	Policy and Orchestration Implications	33
6.5	Implemented Variants	34
7	DYNAMOS Evaluation	37
7.1	Bulk Retrieval Results	37
7.2	Additional Scenario Checks	38
7.3	Time to First Result	38
7.4	Resource Usage	39
7.5	Failure Modes and Robustness	40
8	Discussion	41
8.1	RQ1: Transport Suitability	41
8.2	RQ2: Integration with Policy-Aware Orchestration	42
8.3	RQ3: Case-Study Impact	42
8.4	Comparison with Existing Literature	43
8.5	Threats to Validity	43

8.5.1	Transport benchmark threats	43
8.5.2	DYNAMOS benchmark threats	44
9	Conclusion	45
9.1	Summary of Findings	45
9.2	Contributions	46
9.3	Limitations	46
9.4	Future Work	46
A	Benchmark and Implementation Details	49
A.1	Benchmark Preset Definitions	49
A.2	Full Run Matrix	50
A.3	Run Provenance	51
A.4	DYNAMOS Failure-Mode Notes	52
A.5	Detailed Clean-Matrix Results	53
A.6	Resource Utilisation Details	55
	References	57
	Acronyms	61

List of Figures

6.1	DYNAMOS bulk retrieval path used for RQ2. The implementation changes the bulk data path after policy approval. Chunking starts at the SQL query service and must be preserved through the generated job, agent path, and gateway response to avoid rebuilding one complete result table before the client sees progress.	32
6.2	Logical chunk metadata shared by all chunk-aware variants. The same envelope travels as repeated unary calls, gRPC stream messages, or RabbitMQ stream records depending on the variant.	33
7.1	Average completion time for the local DYNAMOS bulk retrieval benchmark. Chunking is the dominant improvement: all chunk-aware variants scale better than classic unary as the returned row count grows.	37

List of Tables

2.1	Concept matrix for the communication mechanisms considered in this thesis.	8
4.1	Scenario-to-property mapping used in the RQ1 benchmark.	18
4.2	Rationale for the main experimental parameter values.	18
4.3	Primary software versions and sources used for reproducibility. . .	21
5.1	Single-node synthetic-clean summary across six transports, averaged over ten repetitions, four payload sizes, and four concurrency levels.	25
5.2	Continuous steady-rate scenario: single-node results averaged over ten repetitions (1 KiB payload, 1,000 msg/s target). Throughput in msg/s; latency, inter-arrival, and jitter values in milliseconds. . . .	27
5.3	Backpressure scenario: single-node results averaged over ten repetitions (1 KiB, concurrency 8, 4,000 msg/s target, 2 ms sink delay). Throughput in msg/s; latency, inter-arrival, and jitter values in milliseconds.	27
5.4	Recovery scenario: single-node results averaged over ten repetitions (4 KiB, concurrency 8, 2,000 msg/s, transformer restart at 3 s). Throughput values are in msg/s; time, latency, and drain values are in milliseconds. Sustained recovery is the first five complete one-second buckets at or above 80% of target. Dashes indicate that the value was not observed.	28
5.5	Directional consistency between prior benchmark findings and this benchmark. Only orderings and conditions are compared, not absolute values.	30
6.1	DYNAMOS implementation variants and the architectural boundary changed by each variant.	34
6.2	DYNAMOS services modified on the streaming implementation branch (commit 923b018) relative to main (commit a2d9531). The orchestrator and policy enforcer carry the selected transport, but their policy decision and job-composition responsibilities remain unchanged.	35
7.1	Average completion latency for DYNAMOS bulk retrieval. All cells completed 20/20 valid repetitions.	37

7.2	Focused DYNAMOS scenario checks beyond the primary single-provider <code>dataThroughTtp</code> benchmark.	38
7.3	Time-to-first-result and partial result behaviour. Classic unary has no partial results, so first result equals completion.	39
7.4	Peak sampled resource usage during DYNAMOS bulk retrieval. CPU is peak total millicores; memory is peak total MiB across sampled Kubernetes namespaces.	40
A.1	Benchmark presets used in the current transport study.	49
A.2	Execution scope of the current benchmark corpus.	50
A.3	Repository, script, and result locations used for thesis reproducibility.	51
A.4	Failure modes observed or accounted for during the DYNAMOS benchmark work.	52
A.5	Synthetic bulk-transfer throughput by message size, averaged over ten repetitions and concurrency levels 1, 4, 8, and 16.	53
A.6	Detailed synthetic-clean message throughput (msg/s) by payload size and concurrency, averaged over ten repetitions.	53
A.7	Detailed synthetic-clean data throughput (MiB/s) by payload size and concurrency, averaged over ten repetitions.	54
A.8	Detailed synthetic-clean p_{95} latency (ms) by payload size and concurrency, averaged over ten repetitions.	54
A.9	Continuous scenario CPU usage by role (avg./peak %).	55
A.10	Recovery scenario CPU usage by role (avg./peak %).	55
A.11	Recovery scenario memory usage by role (avg./peak MiB).	55

Acknowledgements

I would like to thank my supervisors, Alexandros Koufakis and Dr Ana Oprescu, for their guidance throughout this thesis project. Their feedback, questions, and advice helped me refine both the technical work and the direction of the research. I am also grateful to the Complex Cyber-Infrastructure group at the University of Amsterdam for providing a supportive environment in which to conduct this work.

Finally, I would like to thank my friends and family for their encouragement, patience, and support throughout the project. Their confidence in me was especially valuable during the more demanding stages of the research and writing process.

Introduction

1.1 Context

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organisations. In such settings, data exchange must adhere to legal agreements, privacy regulations, organisational policies, and infrastructure boundaries. These constraints are common in domains such as healthcare, scientific research, public-sector infrastructure, and cross-organisation analytics, where unrestricted data exchange is not acceptable.

Microservice-based middleware is a natural fit for these environments because individual data exchange steps can be deployed, isolated, and scaled independently. However, the communication layer becomes a central design concern. A policy-aware system must not only move data efficiently; it must also preserve the control path that decides which parties may exchange data, where computation may run, and which execution shape is allowed. Communication mechanisms that improve throughput but obscure policy boundaries or complicate orchestration are therefore not automatically suitable.

Streaming communication is attractive in this setting because data can be transferred and processed incrementally. A producer does not need to materialise a complete result before the consumer can begin work, and a client can observe progress before the final result is available. This can reduce peak memory usage and improve completion time for large result sets. At the same time, streaming introduces long-lived flows, buffering choices, backpressure behaviour, replay semantics, and failure modes that are more complex than simple request-response calls.

1.2 Case-Study System

Studying these tradeoffs empirically requires a system in which they actually collide, so the case-study system was selected against explicit criteria rather than assumed upfront. A suitable system must (i) evaluate data-sharing policies before execution and enforce the outcome architecturally, (ii) compose its execution path dynamically rather than through a fixed pipeline, (iii) mediate communication across independent administrative domains, and (iv) be open source and modifiable so that alternative data paths can be implemented and

measured. Several classes of systems satisfy some of these criteria. Dataspace connectors such as the IDS Dataspace Connector implement policy-based, usage-controlled data exchange, but their deployments typically use a fixed connector topology rather than execution paths composed per request [18, 25]. Sidecar-based runtimes such as Dapr unify heterogeneous communication mechanisms behind one interface, but policy-driven orchestration of the execution path is not part of their core model [10]. Stream-processing platforms handle continuous data natively but are not built around per-request policy approval. None of these classes is inherently unable to support the studied combination; they simply do not match it as directly.

DYNAMOS combines all four criteria in one implementation and is therefore used as the case-study system. Stutterheim [37] and Stutterheim et al. [38] describe DYNAMOS as a dynamically adaptive microservice-based middleware for data exchange systems. DYNAMOS implements recurring data-sharing archetypes, such as Compute-to-Data and Data-through-a-Trusted-Third-Party, by generating short-lived service chains after policy evaluation. This makes it a useful case because it combines the concerns that make streaming difficult: policy approval, dynamic orchestration, sidecar-mediated agent communication, generated jobs, and large intermediate results.

The original DYNAMOS data path is built primarily around unary request-response transfers. That model is simple and compatible with generated jobs, but it becomes limiting when large tables or continuous outputs must pass through several services. A full-result response can be buffered repeatedly inside the SQL service, generated job chain, agent path, and gateway before the client sees any progress. The case study therefore asks whether streaming-style transfer can improve scalability while keeping the policy and orchestration model intact.

1.3 Problem Statement

The problem addressed in this thesis is that policy-aware data exchange systems need a communication model that can move large or continuous result sets without repeatedly materialising complete responses, while still preserving policy-driven orchestration and diagnosable execution boundaries. In DYNAMOS specifically, this means changing the bulk data path after approval, not replacing the policy decision point or the generated job model.

The design space is not a simple choice between “streaming” and “not streaming”. Direct gRPC streaming, bounded batched unary transfer, and brokered event streams make different tradeoffs. Direct streaming can reduce overhead when producer and consumer lifetimes are coupled. Batching can retain request-response compatibility while amortising per-message cost. Brokered streams can add temporal decoupling, replay, and retained records, but they introduce broker overhead and configuration sensitivity. The thesis therefore evaluates streaming communication as a set of architectural tradeoffs rather than as a single technology switch.

1.4 Research Questions

The main research question addressed in this thesis is:

Main RQ: How can streaming communication be integrated into dynamically composed, policy-aware microservice-based data exchange systems while preserving compliance-oriented orchestration and improving scalability and performance?

To answer this question, the following sub-research questions are defined:

- **RQ1:** Which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems?
- **RQ2:** How can streaming communication be integrated into a policy-aware microservice architecture in a way that preserves dynamic service composition, policy-driven orchestration, and isolation guarantees?
- **RQ3:** What is the impact of the implemented streaming data paths on scalability, performance, and robustness compared to a traditional request-response approach in the DYNAMOS case study?

1.5 Approach and Contributions

This thesis uses a mixed-methods approach that combines literature analysis, controlled benchmarking, prototype implementation, and in-system validation. First, existing communication mechanisms and evaluation practices are reviewed to identify candidate technologies and relevant comparison dimensions. Second, a controlled Kubernetes benchmark evaluates six transport mappings under clean transfer, continuous flow, backpressure, and recovery scenarios. Third, the findings are used to implement three chunk-aware DYNAMOS data paths: chunked unary transfer, intra-job gRPC streaming, and RabbitMQ Streams at the agent boundary. Finally, these implementations are evaluated inside DYNAMOS on realistic bulk retrieval workloads and checked against multiple archetype and provider shapes.

The contributions are fourfold. First, the thesis provides a controlled comparison of direct and brokered streaming-compatible transport mappings for microservice data exchange. Second, it identifies chunk preservation as the key architectural requirement for improving large-result transfer in a policy-aware generated workflow. Third, it implements and evaluates three chunk-aware data paths in DYNAMOS. Fourth, it documents the benchmark scope, parameter rationale, environment, provenance, and failure-mode notes needed to reproduce and interpret the results.

1.6 Outline

Chapter 2 introduces policy-aware data exchange systems, the DYNAMOS case study, and the communication mechanisms considered in the thesis. Chapter 3 positions the work against literature on streaming communication, adaptive microservice middleware, policy enforcement, resilience, and performance evaluation. Chapter 4 describes the research design, benchmark setup, parameter rationale, validation strategy, and ethical considerations. Chapter 5 presents the controlled transport benchmark. Chapter 6 describes the DYNAMOS streaming integration. Chapter 7 reports the DYNAMOS-specific performance results.

Chapter 8 maps the findings back to the research questions and discusses limitations. Chapter 9 summarises the conclusions and future work.

Background

This chapter introduces the technical context needed to understand the thesis. It first describes policy-aware data exchange systems as the broader class of systems studied here, then introduces DYNAMOS as the case-study system, and finally summarises the candidate communication mechanisms evaluated in the benchmark.

2.1 Policy-Aware Data Exchange Systems

Policy-aware data exchange systems coordinate data movement or computation between parties that do not share a single administrative domain. Their policies may specify who may access data, where computation may run, which party may see intermediate results, and which component is trusted to coordinate the exchange. This makes the communication layer part of the compliance story: a transfer mechanism must preserve clear execution boundaries, not only deliver bytes quickly.

Many such systems can be described through data-sharing archetypes. These archetypes identify where data is located, where computation executes, and which party controls the exchange. Compute-to-Data keeps computation near the provider, while Data-through-a-Trusted-Third-Party uses an intermediary to coordinate or process the exchange. These archetypes are often implemented as bounded jobs, but the archetype definitions do not inherently require all data to be materialised as one response. As long as control and trust boundaries remain explicit, an approved exchange can also be executed incrementally.

2.2 DYNAMOS as Case Study

DYNAMOS [37, 38] translates a policy-approved request into a generated microservice chain. Agents and sidecars mediate communication between administrative domains, while short-lived generated jobs execute the selected data-sharing archetype.

DYNAMOS is a useful case study because it combines three concerns that are usually studied separately. First, it has a policy-controlled orchestration path that decides which exchange is allowed. Second, it creates ephemeral execution units rather than relying on one static pipeline. Third, it can move large intermediate results through several services and agent boundaries. These

properties make streaming integration more demanding than adding a stream API to a fixed producer-consumer pair.

The Data-through-TTP archetype is the main validation path in this thesis. In that archetype, a trusted intermediary coordinates data exchange between parties and enforces the approved execution shape. Conceptually, this resembles brokered or intermediary-mediated communication in streaming microservice architectures. Coviello et al. [4], for example, evaluate real-time streaming analytics pipelines in which an intermediary communication layer enables flexible many-to-many communication. DataXc does not address policy-driven data sovereignty, but it supports the idea that intermediary-mediated streaming can be technically practical.

2.3 Candidate Communication Mechanisms

The thesis considers direct RPC-style transfer, bounded batching, and brokered event streams. These mechanisms are not interchangeable: they differ in coupling, replay, ordering, flow control, operational cost, and fit with generated policy-approved jobs.

2.3.1 gRPC streaming

The official gRPC documentation [13] describes four RPC styles: unary, server-streaming, client-streaming, and bidirectional streaming. For streaming calls, message ordering is preserved within each individual RPC, while deadlines, cancellation, metadata, and synchronous or asynchronous APIs are supported by the runtime.

For microservice chains with point-to-point data transfer, client-streaming is a natural candidate because it lets a sender incrementally push chunks or records to a downstream consumer. Bidirectional streaming becomes relevant when the receiver should provide progress feedback, pacing information, or explicit acknowledgements during transfer. The main advantages are low protocol overhead, reuse of protobuf contracts, and no broker layer. The disadvantages are temporal coupling between endpoints, no default durable replay, and application-level responsibility for backlog and crash recovery. Roohitavaf et al. [32] show that carefully designed asynchronous gRPC streaming can support efficient ordered delivery, making it a strong baseline rather than merely the status-quo option.

2.3.2 Apache Kafka

Apache Kafka is an event-streaming platform built around append-only logs, topics, partitions, retention, and replay [1]. Dobbelaere and Sheykh Esmaili [5] describe Kafka’s log abstraction as a strong fit when temporal decoupling, repeated reads, and high-throughput retention matter. Its disadvantages are operational footprint, partition-local rather than global ordering, and a non-trivial tuning burden around acknowledgements, partitioning, batching, and replication.

2.3.3 RabbitMQ and RabbitMQ Streams

Dobbelaere and Sheykh Esmaili [5] position RabbitMQ as a routing-oriented messaging system with flexible exchanges and acknowledgement support. RabbitMQ Streams add an append-only, non-destructive stream abstraction that supports offset-based reads and replay [28]. This makes RabbitMQ useful as both a classic queue-oriented broker and a stream-log candidate. The limitation is that stream mode introduces its own replication, retention, and partitioning tradeoffs, and classic RabbitMQ queue semantics are not optimised for replay-heavy logs.

2.3.4 NATS with JetStream persistence

The NATS documentation [39–41] presents core NATS as a lightweight connective technology for distributed systems, while JetStream adds persistence, replay, replication, and consumer state. NATS is therefore interesting as a smaller operational alternative to Kafka and RabbitMQ. A key complication is that some comparative literature still discusses the older NATS Streaming layer rather than JetStream; Sharvari and Sowmya Nag [33] are useful historically, but their results do not map perfectly to current JetStream behaviour.

2.4 Concept Matrix

The matrix shows why the thesis does not treat the fastest clean transport as automatically best. Direct gRPC transfer is attractive inside generated jobs where endpoint lifetimes are naturally coupled. Chunked unary is attractive when the existing request-shaped architecture should be preserved. Brokered streams are attractive when decoupling, replay, and independent consumption matter more than local latency.

Table 2.1: Concept matrix for the communication mechanisms considered in this thesis.

Mechanism	Architectural fit	Main strengths	Main limitations	Thesis role
Per-message unary gRPC	Tight request-response; no replay by default; ordering is per request	Simple, debuggable, and compatible with existing RPC APIs	High per-message overhead for bulk transfer; no partial progress for one large response	Lower-bound baseline for request-response transfer
Batched / chunked unary gRPC	Request-shaped but chunked; caller pacing and per-chunk sequence metadata	Amortises overhead while preserving request-style integration	Requires application-level chunk metadata and reassembly	Main DYNAMOS implementation candidate
gRPC client streaming	Tight long-lived stream with transport flow control; no durable replay by default	Low protocol overhead and natural incremental transfer	Couples endpoint lifetimes; recovery and replay remain application responsibilities	Direct-transfer benchmark winner and in-pod optimisation candidate
RabbitMQ classic queues	Broker-decoupled queue delivery with acknowledgements; durability possible	Existing DYNAMOS fit, temporal decoupling, and mature routing	Classic queues are not replay-oriented stream logs	Existing agent-boundary model
RabbitMQ Streams	Broker-decoupled append-only retained stream with offset-based reads	Replay, non-destructive reads, and offset-based recovery	Extra broker overhead and stream-specific retention and replication tuning	Distributed-boundary candidate
NATS JetStream	Broker-decoupled durable streams and consumers; pull or push consumption	Lightweight deployment, replay, and flexible consumer modes	Ack and fetch configuration strongly affect duplicate behaviour and latency	Broker comparison and tuning-sensitive alternative
Kafka	Broker-decoupled durable partitioned log with consumer offsets and batching	Mature event-stream ecosystem and retention model	Operational footprint; partitioning and replication tune performance heavily	Durable log comparison point

Related Work

This chapter reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

3.1 Streaming communication technologies and interaction models

Policy-aware data exchange systems constrain communication choices because the data path has to remain compatible with orchestration and compliance boundaries. DYNAMOS, the case-study system introduced in Chapter 2, is deployed on Kubernetes, uses gRPC for inter-service communication, and uses RabbitMQ for asynchronous messaging [37, 38]. This constrains the case-study design space to technologies that are compatible with containerised microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. Stutterheim [37] identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity.

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request-response interactions. Starck and Ghofrani [35] show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to become unevenly distributed across replicas. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and

ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualisation overhead, with measurable effects on latency and resource utilisation in containerised environments [34].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [17]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes integration [26]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasise that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [4]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. Existing studies are still useful methodologically because they emphasise identical workloads, common measurement points, latency distribution, throughput, resource pressure, and recovery behaviour as recurring benchmark dimensions [3, 16, 20, 30]. The gap addressed in this thesis is therefore not the invention of a new benchmarking methodology, but the application of those dimensions to dynamically composed, policy-driven middleware with ephemeral execution units.

3.2 Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [37, 38]. These chains are short-lived by design, which improves isolation and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. Figueira and Coutinho [10] show that sidecars can support runtime adaptation of communication behaviour in self-adaptive Kubernetes architectures, and discuss sidecar runtimes such as Dapr that unify service invocation, pub/sub,

and state management behind a protocol-agnostic interface. This line of work establishes that heterogeneous communication paradigms, including streaming and messaging, can be hidden behind a stable sidecar interface. What it does not address is dynamic chain composition driven by policy evaluation: sidecar runtimes assume relatively static service topologies, so the combination of streaming transfer with ephemeral, policy-generated execution chains remains open. That combination is precisely the setting of this thesis.

3.3 Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [37, 38]. In the current architecture, policy enforcement is implicitly scoped to bounded request-response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

The access-control literature has a name for this distinction. Park and Sandhu [27] introduce usage control (UCON) as a generalisation of access control in which authorisation is not a one-time decision at request time: decisions can be re-evaluated during use, attributes can change while access is ongoing, and access can be revoked mid-use. Hariri et al. [15] extend this model with an architecture for continuous authorisation in which policy re-evaluation is triggered by attribute changes during an active session. In the data-exchange domain, dataspace connectors such as the IDS Dataspace Connector operationalise usage-control policies for inter-organisational transfer [18]. This vocabulary matters for streaming middleware because it separates two enforcement models: pre-decision enforcement, where a request is approved once before execution, and ongoing enforcement, where an active transfer remains subject to re-evaluation.

Neumaier et al. [25] make the same distinction concrete for decentralised dataset exchange, proposing continuous monitoring and runtime validation against machine-readable policies. While their work does not explicitly target streaming RPC, the distinction between one-time policy clearance and ongoing compliance monitoring becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiyagarajan et al. [42] present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication, and RBAC. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how usage-control-style ongoing enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing

streaming into policy-driven middleware such as DYNAMOS, and they motivate the explicit enforcement-model boundary drawn in Chapter 6.

3.4 Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request-response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

The most directly relevant prior work is fault-recovery benchmarking. Vogel et al. [43] benchmark fault recovery in Flink, Kafka Streams, and Spark Structured Streaming by injecting worker failures in a cloud-native testbed and measuring recovery behaviour over a fixed observation window. They argue that recovery has been under-measured relative to clean throughput, and that meaningful recovery evaluation requires several complementary measures: the time until output resumes, the time until throughput is restored to its pre-failure level, the backlog accumulated during the outage, and the correctness cost of replay under at-least-once delivery. This thesis adopts that structure at the transport level rather than the framework level: the recovery scenario in Chapter 5 injects a component restart and reports resumption time, throughput restoration, accumulated backlog, and post-failure duplicates and ordering violations per transport mapping.

Beyond recovery benchmarking, reactive microservice architectures emphasise asynchronous communication, non-blocking execution, and fault isolation as resilience mechanisms under variable load [29], and Guntupalli and Vamshi [14] show that event-driven architectures with messaging systems can mitigate bottlenecks under sustained data rates. From a systems perspective, Gan and Delimitrou [12] demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments, so resilience and scalability issues often emerge from inter-service communication rather than computation.

Existing literature therefore provides both principles (fault isolation, explicit flow control) and measurement practice (fault-injection benchmarking with multi-dimensional recovery metrics), but applies them either to request-oriented architectures or to stream-processing frameworks with static topologies. There is limited work examining how these mechanisms behave for transport-level streaming in dynamically composed microservice chains.

3.5 Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction styles in terms of throughput, latency, and resource usage. Studies focused specifically on gRPC streaming show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [32,

34]. Complementary protocol-level comparisons between REST and gRPC further establish that binary, multiplexed transports outperform request-response HTTP under load, which provides additional context for choosing gRPC as the baseline communication layer [22, 31].

Scalability in containerised streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerised microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [30]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [35].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

3.6 Research Gap

A substantial body of work addresses streaming technologies in distributed systems. Many studies focus on performance characteristics of stream processing frameworks such as Apache Spark, Flink, and Storm [11, 20, 23]. However, most research on comparing different communication approaches evaluates either traditional request-response patterns or compares streaming frameworks in isolation. When work explicitly compares brokers with RPC mechanisms, it often focuses on unary request-response communication rather than streaming semantics [4, 17, 21, 26]. No existing work found for this thesis provides a direct empirical comparison between gRPC streaming and broker-based streaming that accounts for long-lived connections, flow control, and backpressure handling in realistic microservice deployments.

Architectural integration and policy enforcement in streaming contexts are also addressed by relatively little prior work. DYNAMOS represents a policy-aware, adaptive approach to microservice composition. Stutterheim [37] and Stutterheim et al. [38] describe DYNAMOS and its policy-driven architecture, while related work touches on self-adaptation, policy-aware data exchange, usage control, and gRPC security [10, 25, 27, 42]. However, these studies do not examine how streaming communication affects policy enforcement, service composition, or middleware behaviour in an adaptive system. The question of how to integrate streaming semantics into policy-driven middleware while preserving its adaptive capabilities therefore remains largely unexplored.

Finally, performance evaluation of streaming systems is well represented in the literature [20, 23, 30, 34, 43], but these studies tend to focus on throughput and latency in static stream-processing systems. What is missing is an evaluation of how streaming communication affects microservice architectures where services are dynamically composed into policy-approved execution paths. The interaction between streaming semantics, service replication, load balancing,

and adaptive middleware components motivates the empirical focus of this thesis.

Methods

This chapter describes the research design used to answer the research questions. The thesis combines a literature-backed technology comparison, a controlled transport benchmark, and an implementation-and-evaluation cycle inside DYNAMOS. The purpose is not only to measure which transport is fastest in isolation, but also to determine which communication model fits policy-aware generated workflows.

4.1 Research Design

The research unfolds in three phases, each aligned with one sub-research question. RQ₁ is addressed through a literature-backed comparison and a controlled Kubernetes benchmark of candidate transport mappings. RQ₂ is addressed by designing and implementing chunk-aware data paths inside DYNAMOS while preserving its existing policy and orchestration path. RQ₃ is addressed through an in-system DYNAMOS benchmark that compares the implemented variants against the classic unary baseline under realistic bulk retrieval workloads.

4.2 Transport Benchmark Design

4.2.1 Methodological positioning and scope

The benchmark design adapts recurring evaluation principles to the specific constraints of DYNAMOS rather than copying a single prior protocol. The controlled multi-stage pipeline follows the style of prior streaming-pipeline evaluations such as DataXc [4]; identical business logic across mechanisms follows comparative benchmarking practice for distributed systems, where Karimov et al. [20] require the same workload definitions and measurement points across systems under test, and Henning and Hasselbring [16] define scalability as the rate of throughput growth per added resource unit. The emphasis on throughput, latency, concurrency, resource pressure, and recovery reflects recurring evaluation dimensions in streaming and IPC (Inter-process communication) studies [3, 30].

The aim is not to identify a universally best mechanism, but to narrow the design space for policy-aware data exchange systems, instantiated here in DYNAMOS, by asking which communication model best fits large, ordered, and

concurrent inter-service transfer under realistic operational constraints. The completed corpus evaluates six transport mappings against the same pipeline, the same payload definitions, and the same correctness criteria, and the results are interpreted in terms of DYNAMOS integration. The sixth mapping is batched unary gRPC, in which one unary request carries multiple logical records. The chapter body keeps the main comparison compact: the completed single-node clean matrix, three focused operational scenarios, and the resulting architectural interpretation. Full preset definitions and detailed per-case tables remain in the appendix.

4.2.2 Common testbed

All candidate mechanisms were evaluated in the same Go-based microservice chain deployed on Kubernetes: *Producer* $\rightarrow$ *Transformer* $\rightarrow$ *Sink*. The services ran in a dedicated namespace using the same manifests, the same payload schema, and the same transformation logic. Resource budgets were controlled through Kustomize overlays so that the comparison remained centered on the communication layer rather than on application-level differences [16, 20].

The transport mappings were also kept as close as possible to one another. Client-streaming, per-message unary, and batched unary gRPC provide the direct service-to-service reference points. Kafka preserves per-flow ordering through keyed partitioning. RabbitMQ uses streams rather than classic queues. NATS uses JetStream with asynchronous publishing and explicit consumer acknowledgements. The brokered mappings were tuned for batched or asynchronous sending rather than for one blocking publish acknowledgement per record, so the comparison remains centered on transport behaviour rather than on intentionally slow acknowledgement patterns. Batched unary keeps the same logical messages as the other transports, but groups them into bounded unary requests so that the sink still measures logical records rather than RPC calls.

The service logic remained intentionally lightweight. The producer emitted synthetic or replayed records, the transformer applied a deterministic operation, and the sink recorded latency, pacing, and correctness summaries. That kept the benchmark focused on transfer behaviour, buffering, and recovery rather than on domain-specific computation [4, 30].

4.2.3 Fairness considerations

Direct and brokered transports are not symmetric, and the comparison is only meaningful if those asymmetries are explicit. Four points define what the comparison does and does not equalise. First, brokered mappings pay an extra network hop and broker-side persistence by design; this is not an experimental artifact but the cost of the durability and decoupling being evaluated, so the results are interpreted as a comparison between architectural classes rather than a race between equivalent implementations. Second, no broker pays replication cost: all brokers run single-node with replication factor one, so none is penalised for redundancy the others do not provide. Third, publish configurations were chosen to avoid strawman setups: all brokered mappings use asynchronous or batched publishing rather than one blocking acknowledgement per record. NATS consumption uses explicit pull consumers with a bounded fetch batch, a 500 ms maximum fetch wait, and explicit acknowledgements,

because duplicate behaviour in JetStream is sensitive to the acknowledgement window. Fourth, everything else is held constant: the same services, payload schema, transformation logic, namespace, resource overlays, and measurement points apply to every mapping. Broker persistence and retention settings are documented in Appendix A.1. Given these constraints, band-level conclusions (direct versus brokered) are robust, while small differences between individual brokers are treated as configuration-sensitive rather than decisive.

4.2.4 Evaluation dimensions

The literature consistently treats streaming evaluation as a multi-dimensional problem rather than a single throughput race [3, 4, 30]. This study follows that structure. Five dimensions matter most here: transfer efficiency, latency and pacing stability, overload behaviour, recovery correctness, and operational cost. A transport that leads the clean matrix but degrades badly under backpressure or restart is not automatically the best fit for DYNAMOS. The benchmark therefore keeps those dimensions visible throughout the chapter.

4.2.5 Scenario set and scenario-to-property mapping

The completed RQ1 study uses four scenario families: a clean bulk-transfer matrix, a steady-rate continuous scenario, a slow-consumer backpressure scenario, and a forced-restart recovery scenario. This scenario set is not presented as a new benchmark methodology. It adapts recurring concerns from stream-processing and microservice evaluation literature, namely sustained transfer, latency stability, overload response, recovery after interruption, and correctness under at-least-once delivery [3, 20, 30, 43].

The recovery scenario in particular follows the fault-injection design of Vogel et al. [43], who benchmark fault recovery in stream processing frameworks by killing a worker mid-run and observing a fixed post-failure window. The recovery metrics defined in Chapter 5 operationalise their measurement dimensions at the transport level: time to first post-failure message corresponds to recovery latency, the sustained-throughput criterion corresponds to throughput restoration, recovery debt and backlog drain correspond to accumulated lag and catch-up time, and post-failure duplicate and ordering counts validate delivery guarantees under at-least-once semantics. The only thesis-specific choices are parameterisations of these established measures, and their values are justified in Table 4.2. These properties are relevant to DYNAMOS because generated workflows may transfer large intermediate data, may contain stages with unequal processing rates, and may be restarted or redeployed as part of normal Kubernetes operation.

4.2.6 Experimental factors

The main independent variables are payload size, concurrency, target rate where applicable, sink slowdown in the backpressure case, and explicit failure injection in the recovery case. Concurrency denotes the number of parallel logical workers inside one benchmark run, not the number of producer Kubernetes jobs. In brokered modes, corresponding transformer and sink workers consume the worker-specific streams, subjects, or partitions. Resource budgets are held

Table 4.1: Scenario-to-property mapping used in the RQ1 benchmark.

Scenario	Primary property	Why it matters for DYNAMOS
Clean bulk transfer	Protocol overhead, sustained throughput, multi-hop transfer efficiency	Large dataset exchange should remain efficient when data moves incrementally through a composed service chain.
Continuous steady-rate flow	Latency stability, inter-arrival regularity, pacing precision	Long-lived flows matter when downstream services depend on a predictable delivery rhythm.
Slow-consumer backpressure	Queue build-up, overload propagation, throttling versus delay accumulation	DYNAMOS pipelines can contain stages with unequal service rates, so overload handling must remain interpretable.
Forced-restart recovery	Reconnect behaviour, duplicates, ordering preservation, recovery time	Ephemeral microservices and restarts are normal in Kubernetes-style deployments.

constant through Kubernetes overlays so that performance differences reflect the transport mapping rather than an uneven infrastructure allocation. This keeps the experiment matrix broad enough to expose real tradeoffs without turning the chapter into an exhaustive lab log [3, 33].

Table 4.2: Rationale for the main experimental parameter values.

Parameter	Values	Rationale
Synthetic payload size	256 B, 1 KiB, 4 KiB, 16 KiB	Covers small metadata-like messages through larger serialised records without turning RQ1 into a file-transfer benchmark. The range exposes framing and broker overhead before the DYNAMOS-specific benchmark tests larger real chunks.
Synthetic concurrency	1, 4, 8, 16 logical workers	Probes single-flow overhead and moderate parallelism on the local eight-core benchmark host. Values above 16 would mainly stress the local node scheduler rather than the transport choice.
Continuous target rate	1,000 msg/s	Creates a steady flow that all transports can plausibly sustain, so the comparison emphasises pacing, tail latency, and jitter rather than failure to reach an unrealistic target.
Backpressure target and sink delay	4,000 msg/s with 2 ms sink delay	Intentionally overloads the sink so queue growth, throttling, and latency accumulation become visible.
Recovery restart	Transformer restart after 3 s, fixed 10 s recovery window	Injects failure after initial warm-up while leaving enough post-failure time to compare first message, sustained recovery, duplicates, ordering, and backlog drain.
Sustained-recovery threshold	80% of target over 5 consecutive 1 s buckets	Requires stable restored progress rather than one lucky message. The threshold is a parameterisation of throughput restoration [43]; the transport ordering in the recovery table is driven by order-of-magnitude differences and does not hinge on the exact percentage.
DYNAMOS row counts	50k, 100k, 250k rows	Exercises result sizes below, around, and well above the historical large-message pain point while remaining practical for repeated local Kubernetes runs.
DYNAMOS chunk size	5,000 rows	Avoids per-row messaging overhead while keeping chunks small enough for partial progress. One 50k-run chunk serialised to about 273.7 KiB, larger than the final RQ1 payload range.
Repetitions	10 transport reps, 20 DYNAMOS reps	Balances run time with enough repetition to catch Kubernetes and broker variability. The thesis reports means and discusses visible spread where it affects interpretation.

4.2.7 Result presentation and appendix boundary

The main text reports one clean-summary table, one compact size-wise clean breakdown, and three focused scenario tables for continuous flow, backpressure, and recovery. Detailed per-case clean-matrix tables remain in Appendix A.5. Resource metrics are used only where the sampling scope is the same across the compared cells. Broker-specific resource traces from exploratory runs are kept as supporting context rather than as a primary ranking signal.

4.3 DYNAMOS Benchmark Method

The primary benchmark uses the `dataThroughTtp` archetype with a single data provider, `UVA`. The request still follows the TTP/SURF compute-provider path defined by the archetype, but the data fanout is intentionally limited to one provider so that the experiment isolates large-result transfer rather than multi-provider coordination.

Each request executes a bulk projection query over the large `PersonenLarge` and `AanstellingenLarge` tables and joins them on `Unieknr`. The result is limited to 50,000, 100,000, or 250,000 rows. The chunked variants use 5,000 rows per SQL chunk, which gives 10, 20, and 50 total chunks respectively. This value is an experimental configuration rather than a claimed optimum: it is large enough to avoid turning the benchmark into a per-row messaging test and small enough to expose partial progress. In the saved 50k-row bulk output, one 5,000-row chunk serialised as a DYNAMOS protobuf `MicroserviceCommunication` is approximately 280,255 bytes, or 273.7 KiB. That is larger than the RQ1 synthetic payload range. This layering is deliberate: the RQ1 results are mechanism-level evidence across a controlled payload range, while the in-system RQ3 measurements carry more weight for the final implementation choice because they use realistic chunk sizes on the real execution path.

Four implementations are compared. Classic unary is taken from the original DYNAMOS `main` branch at commit `a2d9531`. The chunked unary, intra-job gRPC streaming, and RabbitMQ Streams variants are taken from the streaming implementation branch at commit `923b018`. Every reported DYNAMOS cell uses 20 warm repetitions. Runs are accepted only when row counts match the expected limit and canonical content hashes validate the returned content. For chunked variants, the benchmark also requires partial result events before completion.

Two focused scenario checks are added after the primary benchmark. One uses the `computeToData` archetype. The other keeps `dataThroughTtp` but uses two data providers, `UVA`, `VU`. These checks test whether chunk preservation remains compatible when the generated job topology and aggregation path differ from the primary single-provider TTP case.

Resource usage is sampled from Kubernetes once per second across the relevant DYNAMOS namespaces and summarised as peak total CPU millicores and peak total memory in MiB. These sampled peaks are useful for comparing memory pressure, but they may underestimate short instantaneous spikes between samples. The measured request window starts immediately before the benchmark client submits the HTTP request and ends when the final response or final NDJSON event is observed. This means policy handling, orchestration, generated job creation, pod startup, data transfer, and response serialisation

are inside the measured window for both classic and chunked variants. For the classic unary baseline, stale generated jobs are cleaned between repetitions, outside the measured request window (Appendix A.4).

4.4 Validation and Reproducibility

Throughout all phases, the research prioritises reproducibility and transparency. The transport benchmark controls everything except the communication mechanism under test: the same Go-based services, payload schema, transformation logic, Kubernetes namespace, and resource overlays are reused across transport mappings. The DYNAMOS benchmark likewise compares implementations over the same archetype, query shape, row limits, validation criteria, and measurement window.

Synthetic workloads are used rather than unpredictable production traffic. This makes experiments repeatable and allows payload size, concurrency, target rate, sink delay, and failure injection to be controlled. Repetition counts and acceptance criteria are defined per benchmark in Sections 4.2 and 4.3; runs enter the reported corpus only when their correctness checks pass.

The metrics align with established distributed-systems evaluation practice: throughput, data throughput, latency percentiles, pacing, duplicates, ordering violations, recovery behaviour, CPU, and memory. The thesis reports arithmetic means rather than medians because the means include the long-tail runs that occur in local Kubernetes deployments and are therefore the more conservative summary; observed spread and outliers are discussed in the text where they affect interpretation, and per-case ranges are reported for the largest DYNAMOS workloads. Configuration details and extended tables are kept in the appendices so the main chapters remain readable without hiding the evidence needed for reproducibility.

4.5 Experimental Environment

The local benchmark host used an Intel Core i7-9700K CPU with 8 physical cores and 8 logical processors, and 31.9 GiB RAM. The operating environment was Windows with WSL2 Ubuntu 24.04.3, kernel 6.6.87.2-microsoft-standard-WSL2. Kubernetes execution was driven through the repository's kind/kubectrl scripts for the `grpc-stream-single` single-node cluster; the recorded client tooling used Docker Desktop client 29.1.3 and kubectrl client v1.34.1.

4.6 Ethical Considerations

This section evaluates potential ethical concerns and describes plans to address them. The main areas of ethical concern identified are evaluated below.

Human research participants and new data collection. This research does not involve human participants, surveys, interviews, or the collection of new data from people. All evaluation is conducted using synthetic workloads and controlled experimental data generated specifically for performance testing.

Table 4.3: Primary software versions and sources used for reproducibility.

Item	Value
Benchmark repository	[36]
DYNAMOS baseline	Main branch commit a2d9531 [6]
DYNAMOS streaming branch	feature/streaming-implementations commit 923b018 [7]
Go module and libraries	Go 1.25.0, gRPC-Go v1.75.1, nats.go v1.51.0, rabbitmq-stream-go-client v1.8.0, kafka-go v0.4.51
Broker container images	rabbitmq:3-management, nats:2.10-alpine, apache/kafka:3.9.1
RQ1 scripts and results	experiments/grpc-streaming-baseline/scripts/run-k8s-matrix.sh and experiments/grpc-streaming-baseline/results/
Kubernetes overlays	experiments/grpc-streaming-baseline/k8s/overlays/, especially the single-node medium resource overlay

Existing datasets about people. The research does not use, analyse, or combine existing datasets containing personal information. Performance experiments rely entirely on artificially generated data with no relation to real individuals or organisations.

AI technologies and algorithms. While DYNAMOS is a system that could be used to orchestrate AI workloads, this thesis does not design, develop, or deploy AI technologies or machine learning algorithms. The focus is on communication mechanisms in distributed systems, not on AI-specific capabilities.

Cyber security and online privacy. The research addresses communication patterns in distributed systems, which has indirect relevance to security and privacy. However, the thesis does not involve penetration testing, vulnerability research, or deployment in production environments handling sensitive data. All experiments are conducted in isolated development and testing environments under the control of the CCI research group.

The streaming integration preserves DYNAMOS’s existing policy enforcement mechanisms, which are designed to maintain compliance with data-sharing agreements. While the thesis examines how policy enforcement interacts with streaming communication, it does not weaken or bypass existing security controls.

Dual-use technology. DYNAMOS is designed as infrastructure for policy-compliant data exchange in research and organisational contexts. The streaming extensions developed in this thesis are general-purpose communication improvements that enhance scalability and performance. While any communication infrastructure could theoretically be repurposed, the research does not create capabilities specifically designed for surveillance, intrusion, social manipulation, or military applications. The focus remains on improving data-sharing middleware for legitimate scientific and organisational use cases.

4.6.1 Ethical safeguards and compliance

Based on the above assessment, this research falls into a low-risk category with respect to the identified ethical concern areas. Nevertheless, the following safeguards were maintained:

- All experimental deployments used synthetic data with no relation to real individuals or organisations
- Performance testing was conducted in isolated environments that do not interact with production systems or real user data
- Source code and architectural documentation were reviewed to ensure no unintended vulnerabilities were introduced through the streaming integration
- Any deployment recommendations explicitly address the need to maintain existing policy enforcement and security controls

4.6.2 Consultation and approval

If unexpected ethical concerns arise during the research, for example, if collaboration with external organisations introduces access to real data, the Ethical Review Committee will be consulted immediately via the RMS portal for Faculty of Science.

No data collection, deployment in production environments, or experimentation with real user data will occur without prior ethical assessment and approval from the appropriate committees.

Transport Benchmark

This chapter presents the controlled transport benchmark used to answer RQ1. The purpose is to compare candidate communication mechanisms under the same lightweight microservice workload before any mechanism is embedded in DYNAMOS.

5.1 Results

5.1.1 Metric definitions

The benchmark reports the following metrics. The clean, continuous, and backpressure tables use the first group of metrics. The recovery table adds a second group because restart behaviour cannot be described by one number.

- **Message throughput.** Message throughput reports how many messages reach the sink per second. It is the main clean-matrix metric because it shows how much protocol and broker overhead each transport adds for the same logical workload.
- **Data throughput.** Data throughput reports the same transfer rate in MiB/s. It is included because two transports can move a similar number of messages while carrying different effective byte volumes once framing, batching, and message size are considered.
- **Median latency.** Median latency is the typical end-to-end time between producer emission and sink observation. It helps separate transports that are usually fast from transports that only look good in aggregate throughput.
- **p_{95} and p_{99} latency.** These percentiles report tail latency. They matter because broker queues, batching, and consumer polling can keep average throughput high while delaying the slowest messages.
- **p_{95} inter-arrival.** Inter-arrival delay measures the spacing between consecutive messages at the sink. In the continuous and backpressure scenarios it shows whether the sink receives a regular flow or bursty delivery.
- **Jitter.** Jitter summarises variation in inter-arrival timing. A low-jitter transport delivers messages in a more predictable rhythm, which is useful when downstream services depend on steady pacing.

- **Duplicates.** Duplicate counts show how often the sink observes the same logical message more than once. This is important for at-least-once systems because replay, retries, or commit timing can preserve delivery while still requiring idempotent downstream processing. Sink throughput is computed from unique logical messages; duplicate deliveries are counted separately and do not inflate the reported message rate.
- **Ordering violations.** Ordering violations count messages that arrive out of the expected per-flow sequence. This matters for DYNAMOS-style pipelines because some transformations assume that chunks from the same logical stream remain ordered.
- **Pre-failure throughput.** In the recovery scenario, pre-failure throughput confirms whether the transport was near the configured target before the restart. Without this value, a recovery result could be misleading because a transport might appear stable only because it was already under-producing.
- **Recovery-window throughput.** Recovery-window throughput measures delivery during the fixed window immediately after the forced restart. It shows how much useful progress continues while the system is reconnecting or replaying.
- **Post-recovery throughput.** Post-recovery throughput measures the rate after the immediate disruption has passed. It shows whether the transport returns to normal transfer behaviour or remains degraded after the restart.
- **First post-failure message.** This is the time until the first message is observed after the restart. It measures visible resumption, but not full recovery, since a transport can emit one message quickly while still carrying a large backlog.
- **Sustained 80%/5s recovery.** This is the first point at which the sink observes five complete one-second buckets at or above 80% of the target rate. It is stricter than first-message recovery because it requires stable progress rather than a single successful delivery.
- **Recovery debt.** Debt is the number of messages missing from the fixed recovery window relative to the configured target. It represents how much work the transport failed to deliver on time and therefore how much backlog or lost progress remains after disruption.
- **Recovery p_{95} latency.** This is the tail latency of messages observed in the recovery window. It reveals whether recovery is happening through smooth resumed delivery or through delayed backlog bursts.
- **Post-failure duplicates and ordering violations.** These are correctness counts restricted to the part of the run after the restart. Separating them from clean-run correctness makes it visible whether failures trigger replay side effects.
- **Backlog drain and completion.** Backlog drain measures how long it takes for delayed work to clear after the restart. The completion flag records whether the run eventually reached the expected message count.
- **Resource pressure.** CPU and memory measurements are used as supporting evidence where the same sampling scope is available across the compared runs. They indicate whether a transport improves throughput by adding acceptable overhead or by shifting pressure to the broker layer.

5.1.2 Comparison scope

The primary comparison in this chapter is a single-node Kubernetes benchmark across six transport mappings: client-streaming gRPC, per-message unary gRPC, batched-unary gRPC, RabbitMQ Streams, NATS JetStream, and Kafka. Batched unary is the transport-level analogue of the chunked-unary DYNAMOS implementation evaluated in Chapters 6 and 7: both carry a bounded group of logical records per unary request, so the two terms describe the same mechanism at different layers. Single-node means that the pipeline services and broker components are scheduled on one Kubernetes node. This isolates transport overhead without adding cross-node replication or placement effects. Kafka, RabbitMQ Streams, and NATS JetStream can also be deployed as multi-node broker clusters, where partitioning, replication, and placement can change both throughput and resilience [1, 26, 28, 39]. Exploratory clustered broker runs were performed, but they are used as supporting context rather than as primary evidence because they expand the experiment from DYNAMOS communication design into broker-cluster tuning.

All RQ1 tables in the main text report arithmetic means over ten repetitions unless stated otherwise. This follows the preferred reporting style for the thesis. The text still discusses visible spread and non-monotonic cells where they affect interpretation, because Kubernetes and broker timing can introduce outliers that a mean alone would hide.

Table 5.1: Single-node synthetic-clean summary across six transports, averaged over ten repetitions, four payload sizes, and four concurrency levels.

Transport mode	Avg. msg/s	Avg. MiB/s	Avg. p_{95}	Avg. p_{99}	Duplicates	Ordering viol.
Client-streaming	53,164.34	102.49	107.55	133.12	0	0
Batched unary	49,096.11	101.44	51.92	69.27	0	0
Unary	3,066.43	13.09	1.60	23.64	0	0
RabbitMQ Streams	13,648.01	31.58	873.08	912.17	0	0
NATS JetStream	13,336.05	38.20	1,115.43	1,185.77	0	0
Kafka	11,293.92	27.79	1,186.93	1,323.08	105	0

Table 5.1 shows two important results. First, client-streaming and batched unary form the fast direct-transfer band. Client-streaming has the highest average message throughput, but batched unary is close and has lower average tail latency in this matrix. This matters for DYNAMOS because it shows that the decisive property is amortising per-message overhead, not streaming as a label. A request-response mapping becomes competitive when one request carries a bounded batch of records instead of one logical message. Second, per-message unary remains much slower. It has low latency for individual requests because it never builds deep queues, but it pays the full request-response cost for every message and is therefore a lower-bound mapping rather than a realistic bulk-transfer design.

The brokered transports form a separate band. RabbitMQ Streams has the highest average brokered message throughput, while NATS JetStream moves slightly more data per second on average because it performs better in several larger-payload cells (Appendix A.5, Table A.5). The price is substantially higher tail latency, caused by broker persistence, routing, consumer polling, and backlog effects. Ordering violations are absent in the clean matrix. Kafka produces 105 duplicate observations in total across the ten repetitions of the clean matrix;

these do not inflate the throughput number because the sink counts unique logical messages for throughput and records duplicate observations separately. These duplicates are expected behaviour under Kafka’s at-least-once delivery contract rather than a configuration defect: the producer retries failed sends (up to ten attempts with leader acknowledgement), and the consumer group commits offsets periodically every 250 ms, so a retried batch or a commit gap can deliver a record twice [1, 5]. Kafka’s idempotent and transactional modes could remove these duplicates, but they were not enabled because all transports were deliberately run with comparable at-least-once semantics. NATS JetStream records no clean-run duplicates in the evaluated pull-consumer configuration; its duplicate behaviour is sensitive to acknowledgement-window settings, which is why those parameters are fixed in Section 4.2.

The band structure itself follows from the transport architectures. Client-streaming sends every record over one established HTTP/2 stream, so the per-record cost is framing and flow-control accounting rather than a full RPC. Batched unary pays one RPC per bounded batch, amortising the request-response overhead across the records it carries. Per-message unary pays that full overhead for every record and is bounded by request round-trips. The brokered mappings add two network hops (producer to broker, broker to consumer), broker-side persistence, and consumer polling; their higher tails reflect queueing and flush timing inside the broker rather than slower steady-state transfer.

5.1.3 Clean bulk-transfer matrix

The size-wise breakdown is reported in Appendix A.5, Table A.5. It supports the same conclusion as the main clean summary: client-streaming and batched unary remain close across the tested payload range, while per-message unary stays far behind for bulk transfer. Several brokered cells are non-monotonic, so the robust conclusion is the band-level ordering rather than the exact ranking of every broker at every payload and concurrency. The direct gRPC mappings dominate local throughput, with batched unary close enough to client-streaming to justify taking a chunked-unary design into DYNAMOS. The brokered mappings are slower locally, but they still matter architecturally because they provide decoupling and replay semantics that direct streams do not provide by themselves.

5.1.4 Continuous steady-rate scenario

The continuous scenario uses 1 KiB messages and a target rate of 1,000 msg/s. The purpose is not to maximise throughput, but to see how steadily each transport sustains an ongoing flow.

All transports remain near the configured target, so the comparison is mainly about pacing. Unary has the lowest tail latency because each message is acknowledged independently and the producer naturally slows down. Client-streaming also has low tail latency and regular inter-arrival times. Batched unary sustains the target, but its high p_{95} latency and near-zero inter-arrival percentile show bursty delivery: records wait until their batch is full or its timer fires, then arrive together, so an individual record’s end-to-end latency includes its batching delay even though delivery keeps pace overall. That is acceptable for bulk transfer, but less attractive for workloads that need steady

Table 5.2: Continuous steady-rate scenario: single-node results averaged over ten repetitions (1 KiB payload, 1,000 msg/s target). Throughput in msg/s; latency, inter-arrival, and jitter values in milliseconds.

Transport mode	Concur.	Throughput	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	4	975.41	7.80	1.60	0.68
Client-streaming	8	977.28	5.95	1.67	0.71
Batched unary	4	1,050.76	376.15	0.00	13.30
Batched unary	8	976.29	759.42	0.00	19.42
Unary	4	975.72	0.79	2.07	1.15
Unary	8	974.03	1.00	2.10	1.01
RabbitMQ Streams	4	999.97	31.43	2.85	1.27
RabbitMQ Streams	8	1,054.28	44.94	2.77	1.73
NATS JetStream	4	994.95	30.77	2.13	1.00
NATS JetStream	8	984.83	39.57	2.17	1.06
Kafka	4	972.13	355.37	0.01	8.18
Kafka	8	975.11	377.50	0.02	8.17

per-message pacing. Among the brokered transports, NATS and RabbitMQ Streams have moderate tails. Kafka remains burst-dominated with high jitter, which follows directly from its fetch configuration: the consumer returns records only after accumulating 64 KiB or waiting 250 ms, so at a paced 1,000 msg/s flow it delivers in poll-sized groups rather than as a continuous stream.

5.1.5 Backpressure scenario

The backpressure scenario keeps the payload at 1 KiB and concurrency at 8, but adds a 2 ms sink delay while the producer targets 4,000 msg/s. This exposes how each transport behaves once downstream processing becomes the bottleneck.

Table 5.3: Backpressure scenario: single-node results averaged over ten repetitions (1 KiB, concurrency 8, 4,000 msg/s target, 2 ms sink delay). Throughput in msg/s; latency, inter-arrival, and jitter values in milliseconds.

Transport mode	Throughput	Median lat.	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	3,363.09	1,476.52	1,933.46	2.16	0.71
Batched unary	3,444.25	115.80	217.62	1.59	0.57
Unary	2,270.75	0.47	1.06	3.18	1.07
RabbitMQ Streams	3,004.51	1,864.02	2,572.95	2.17	0.71
NATS JetStream	3,039.57	1,796.86	3,142.22	2.16	0.66
Kafka	1,347.23	3,453.50	10,472.11	2.30	0.96

Batched unary is the strongest backpressure result in this run. It achieves the highest realised throughput and keeps the tail much lower than client-streaming and the brokered mappings. Client-streaming, NATS, and RabbitMQ Streams all keep useful throughput under pressure, but they absorb the overload as queueing delay, with median latencies around 1.5–1.9 s and p_{95} latencies around 1.9–3.1 s. Unary gives up throughput earlier and therefore keeps the requests

that complete very low-latency. These outcomes reflect where each transport buffers overload. Per-message unary has implicit backpressure: the producer waits for each response, so it slows to the sink’s rate instead of building a queue. Batched unary bounds in-flight work to one batch per worker, which caps queue depth and therefore the tail. Client-streaming and the brokered mappings keep producing into transport or broker buffers, so the overload surfaces as latency rather than reduced send rate. Kafka is the weakest backpressure result, with the lowest throughput and a p_{95} latency above 10 s: its consumer fetch batching and broker-side backlog compound under sustained overload. It produces 84.5 duplicate observations on average per repetition across the ten repetitions, again without inflating unique-message throughput.

5.1.6 Recovery scenario

The recovery scenario uses 4 KiB messages at 2,000 msg/s and forces a transformer restart after 3 seconds, following the fault-injection design used in stream-processing recovery benchmarking [43]. The table separates the run into pre-failure throughput, a fixed 10 s recovery window, and the post-recovery tail so that restart visibility, sustained recovery, backlog drain, and correctness cost are not collapsed into one ambiguous number; the metric-to-literature mapping is given in Section 4.2.

Table 5.4: Recovery scenario: single-node results averaged over ten repetitions (4 KiB, concurrency 8, 2,000 msg/s, transformer restart at 3 s). Throughput values are in msg/s; time, latency, and drain values are in milliseconds. Sustained recovery is the first five complete one-second buckets at or above 80% of target. Dashes indicate that the value was not observed.

Transport mode	Pre msg/s	Recovery msg/s	Post msg/s	First post-fail	Sustained 80%/5s	Debt	Recovery p_{95}	Post-fail dup.	Post-fail order	Backlog drain	Done
Client-streaming	2,687	653	2,009	0.88	8,500	13,474	1.28	10,462	0	27,769.05	Yes
Batched unary	2,054	1,840	1,831	2,040.29	1,500	3,550	396.51	0	0	24,115.67	Yes
Unary	2,190	1,641	1,901	2,013.24	7,000	5,634	4.69	0	0	24,558.62	Yes
RabbitMQ Streams	1,988	1,610	2,057	324.40	5,500	4,186	2,943.19	910	0	24,242.62	Yes
NATS JetStream	2,154	1,918	216	186.78	5,000	2,753	701.81	0	8	123,704.74	Yes
Kafka	2,359	0	1,965	30,840.90	–	20,000	–	98	0	32,377.34	Yes

Before failure, all transports are close enough to the configured target to make the recovery comparison meaningful. Client-streaming produces the fastest visible resumption, but its current reconnect policy replays the worker stream and therefore creates 10,462 post-failure duplicates on average. Its recovery-window throughput is also low, so the fast first message does not mean the stream has fully recovered. Batched unary and RabbitMQ Streams have a better balance between recovery-window throughput and debt. Batched unary has no duplicate or ordering cost, while RabbitMQ Streams resumes earlier but produces 910 post-failure duplicates and much higher recovery-window tail latency.

NATS JetStream resumes quickly after the restart and records no sink-level duplicates, but the pull-consumer configuration trades aggressive replay for lower post-recovery throughput, eight post-failure ordering violations on average, and a much later backlog-drain point. The ordering violations follow from redelivery semantics: unacknowledged messages from before the restart are redelivered and interleave with newly published messages. Kafka is the clearest negative

case: no messages arrive in the fixed recovery window, first visible recovery is delayed by more than 30 s, and post-failure duplicates still occur. This delay is explained by consumer-group rebalancing rather than data loss. The restarted transformer consumer disappears without leaving the group, so the broker waits for its session to expire (the client library’s default session timeout is 30 s, which this benchmark does not override) before reassigning partitions, and only then does consumption resume. The post-failure duplicates come from redelivery after the last committed offset, consistent with the 250 ms periodic commit interval. RabbitMQ Streams shows the mirror-image behaviour: offset-based resume restarts delivery quickly, but the consumer replays the retained records after its last tracked offset, producing the 910 post-failure duplicates. The recovery table therefore does not select the same winner as the clean matrix. It shows a tradeoff between fast resumption, replay cost, ordering, and backlog drain.

5.1.7 Relation to prior benchmarks

Absolute numbers do not transfer between benchmark studies: hardware, payload definitions, replication factors, persistence settings, and client libraries all differ, and some prior work evaluates NATS Streaming rather than the current JetStream layer. What can be compared is direction and structure: which mechanism class is faster than which, and under which conditions rankings flip. Table 5.5 makes that comparison explicit for the main findings of this chapter.

The one apparent divergence deserves attention because it looks like a contradiction and is not. Survey-level claims that Kafka delivers “significantly higher throughput” than its competitors come from partitioned, batched, scale-out deployments [33]. This benchmark runs Kafka on a single node, with one partition per logical flow, a 5 ms producer batch timeout, and at-least-once acknowledgements. Those are precisely the conditions under which Dobbelaere and Sheykh Esmaili measured RabbitMQ ahead of Kafka and quantified Kafka’s acknowledgement penalty at 50–75% of its best-effort throughput [5]. The benchmark therefore does not contradict prior work; it occupies the corner of the configuration space that prior work identifies as Kafka’s weakest, which is also the corner that matches the DYNAMOS deployment model of per-request flows rather than wide partitioned topics.

The main thesis-specific addition is the batched unary result. The clean and backpressure scenarios show that the improvement is not caused by the word “streaming” alone. The important property is amortising per-message overhead while preserving ordered incremental progress. That observation directly motivates the DYNAMOS implementation strategy in the next chapter.

5.1.8 Implications for the DYNAMOS integration

The benchmark does not point to one transport that should be used everywhere in DYNAMOS. It identifies which mechanisms are promising at which boundary. For local service-to-service transfer, direct gRPC mappings are strongest. Client-streaming gives the highest raw clean throughput, but batched unary is close enough to change the interpretation of the unary result: the weak per-message unary column is not a verdict against request-response communication, it is

Table 5.5: Directional consistency between prior benchmark findings and this benchmark. Only orderings and conditions are compared, not absolute values.

Prior finding	This benchmark	Consistency
Direct asynchronous gRPC streaming outperforms a Kafka-based pipeline for ordered delivery: on average $4\times$ faster, $6\times$ at p_{99} , for 10 KB payloads [32]	The direct gRPC band averages roughly four times the brokered band in the clean matrix; brokered tail latencies are an order of magnitude higher	Agrees in direction and rough magnitude
Direct microservice communication is efficient when the topology is controlled; intermediary-mediated flexibility costs latency and throughput [4]	Same band split between direct and brokered mappings	Agrees
On a single node with one partition per flow, RabbitMQ outperforms Kafka; Kafka improves markedly with partition count; at-least-once acknowledgements cost Kafka 50–75% of its best-effort throughput [5]	RabbitMQ Streams sits above Kafka on one node, with per-flow partitioning and at-least-once semantics	Agrees under matching conditions
Kafka shows significantly higher throughput than RabbitMQ in scale-out, batched configurations; NATS Streaming throughput is comparable to Kafka in some tests and configuration-sensitive [33]	Kafka is the slowest broker here, but the tested setup removes partition parallelism and batching headroom; NATS averages about 18% more messages per second than Kafka	Diverges for Kafka, explained by configuration; agrees in broad order for NATS
No broker dominates across workloads; rankings depend on scenario and configuration [5, 17]	The three brokers stay within a $\sim 21\%$ clean-throughput band, and the ordering shifts per scenario: RabbitMQ Streams leads the clean matrix, NATS leads several large-payload cells, Kafka is weakest under backpressure and recovery	Agrees
Push/pull choice and consumer-controlled flow shape throughput and latency under load [4, 24]	Overload placement differs per mechanism: implicit backpressure in per-message unary, bounded in-flight batches in batched unary, buffer-absorbed overload in streams and brokers	Agrees in direction

a verdict against one-request-per-message transfer. Once multiple records are carried per request, unary becomes a strong bulk-transfer mechanism.

This is why the integration in Chapter 6 focuses on chunk preservation rather than on replacing every boundary with a stream. DYNAMOS has two communication regimes. Inside generated jobs, services already communicate over gRPC, so direct streaming can be used where the sender and receiver lifetimes are naturally coupled. Across agents, DYNAMOS relies on sidecar-mediated RabbitMQ communication, dynamic job readiness, and retry logic. At that boundary, a chunked message model fits the existing architecture better than a single long-lived direct stream.

The implementation hypothesis carried into the next chapter is therefore chunked transfer at existing boundaries, with gRPC streaming as an in-pod optimisation and RabbitMQ Streams as a distributed-boundary candidate rather than as the expected source of a local speedup.

DYNAMOS Integration

This chapter addresses RQ2 by moving from a generic transport benchmark to an implementation inside DYNAMOS. The question is no longer only which communication mechanism is fast. The implementation must also preserve DYNAMOS’ policy approval path, generated job model, sidecar-mediated communication, and isolation between data stewards.

6.1 Motivation

The original DYNAMOS implementation already identifies large message transfer as a limitation. Stutterheim [37] notes that the default maximum size of a gRPC method is around 4 MB, corresponding to approximately 30,000 records in the original experiments, and that the current gRPC implementation uses unary RPCs: one request and one response. The same discussion observes that increasing the message size is theoretically possible but memory-heavy, and explicitly names streaming as the unfinished solution. This makes large-result transfer an appropriate point at which to evaluate streaming techniques inside DYNAMOS rather than only in a synthetic pipeline.

The implementation target is deliberately different from the average-query example commonly used in DYNAMOS demonstrations. An average query is a reduction workload: it may scan many records but returns one scalar result. That tests computation near the data source, but it does not test whether DYNAMOS can transport many aligned rows. Bulk retrieval therefore stresses the part of DYNAMOS that is most affected by full-response buffering. It is also a useful precursor to later distributed learning scenarios, but this thesis first validates whether the core DYNAMOS data path can move large row sets incrementally.

6.2 Execution Path

DYNAMOS separates the control path from the data path. A client request first passes through the gateway, policy evaluation, orchestration, and the relevant agents. Only after that approval and composition step are generated job pods used to execute the request. For the `dataThroughTtp` archetype, the data provider job retrieves rows, the Trusted Third Party (TTP) or SURF-side job

performs the compute-provider part of the chain, and the response eventually returns through the agent and gateway path.

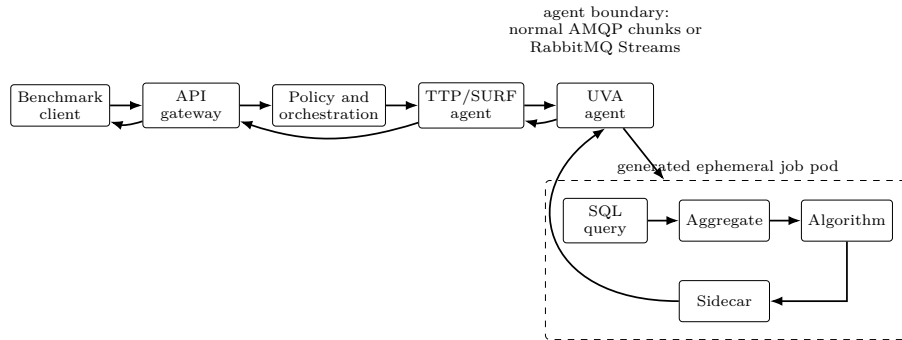


Figure 6.1: DYNAMOS bulk retrieval path used for RQ2. The implementation changes the bulk data path after policy approval. Chunking starts at the SQL query service and must be preserved through the generated job, agent path, and gateway response to avoid rebuilding one complete result table before the client sees progress.

Figure 6.1 shows the boundary that matters for the implementation. In the original path, a large result can be rebuilt several times before the client sees anything: inside the SQL response, inside the generated service chain, inside the agent response, and again at the gateway. The streaming implementation therefore treats chunk preservation as the central requirement. A transport is only useful for this workload if the first rows can leave the provider path before the final rows have been read and serialised.

The implementation should not be a special case for one benchmark request. DYNAMOS supports multiple archetypes and multiple data providers, so a chunk-aware path must keep existing request shapes valid. The primary evaluation uses `dataThroughTtp` with one provider to isolate the large-result bottleneck. RQ3 then checks the `computeToData` archetype and a two-provider `dataThroughTtp` request, because those scenarios exercise different parts of the generated job and aggregation path.

6.3 Common Chunk Model

All chunked variants use the same logical stream model. The SQL query service reads the bulk result in fixed row batches. Each batch carries stream metadata: provider identity, correlation identifier, sequence number, row count, partial or final status, and ordered column names. Listing 6.2 shows the logical chunk envelope carried by every chunked message, independent of the transport that moves it. The ordered-column metadata is important because Go protobuf structs can otherwise expose map fields in nondeterministic order, which would make equal results appear different across transports.

The aggregate and algorithm stages preserve chunk boundaries for non-average bulk workloads. They do not merge all incoming rows into one table unless the request explicitly uses the classic unary baseline. The gateway exposes chunk progress to the benchmark client through NDJSON `providerResult`

```
// Request metadata retained on every MicroserviceCommunication
request_metadata.correlation_id string

// String metadata added to each chunk (streaming branch, 923b018)
stream_provider      string // data steward that produced the chunk
stream_sequence      string // per-provider sequence, starting at 1
stream_rows_processed string // cumulative rows emitted
stream_rows_total     string // expected or final row total
stream_partial        string // "true" until the final chunk
stream_final          string // "true" on the final chunk
stream_batch_id       string // provider and sequence identifier
stream_columns        string // JSON array preserving column order
```

Figure 6.2: Logical chunk metadata shared by all chunk-aware variants. The same envelope travels as repeated unary calls, gRPC stream messages, or RabbitMQ stream records depending on the variant.

events, with partial events before the final event. This keeps the external response shape compatible with the existing gateway while making progress visible.

6.4 Policy and Orchestration Implications

The streaming integration changes the data path, not the policy control path. In usage-control terms [27], DYNAMOS implements pre-decision enforcement: the request is evaluated once, before execution, and the approved generated job is the unit of execution. The implemented chunked variants keep exactly that model. This is not a weakening relative to the original system, because classic unary transfer is enforced at the same single decision point; a result delivered under chunking reflects the agreement at request time, exactly as before. The implementation does not claim that policies are re-evaluated for every chunk or that compliance is continuously monitored during transfer.

Within that unchanged enforcement model, chunking still changes what enforcement *can* do. Three points on the design spectrum are worth distinguishing:

1. **Request-time approval (implemented).** One policy decision authorises the whole transfer. Revocation during transfer is nevertheless already possible at job granularity: the generated job is a Kubernetes resource, and deleting it terminates the data path. Classic unary transfer cannot be interrupted meaningfully at all; the client receives either the complete result or nothing.
2. **Chunk-boundary re-validation (feasible extension).** Because the data path now consists of discrete, sequence-numbered chunks, every chunk boundary is a natural enforcement point. A cheap validity check against the current agreement version before forwarding each chunk would bound the staleness of an agreement change to a single chunk, without a full policy evaluation per chunk. If an agreement changes mid-transfer, the already delivered prefix is well-defined and auditable through the sequence and provider metadata of Listing 6.2.

3. **Continuous authorisation (out of scope).** Full usage-control-style ongoing enforcement [15, 27], for example lease- or capability-based transfer rights that expire unless renewed by the policy engine, would require integrating policy evaluation into the data plane and is treated as future work.

The important observation is that chunk preservation creates the mid-stream enforcement points that neither classic unary transfer nor an opaque long-lived stream provides. With a full-buffered response there is nothing to interrupt or audit until delivery is complete; with an opaque stream a revocation leaves the delivered state undefined. With chunks, a policy change can take effect at the next chunk boundary and leaves an auditable, well-defined delivered prefix. The implemented system stops at point (1), but the mechanism required for point (2) is already in place.

The main orchestration implication is that streaming makes progress observable before completion. That helps diagnose long-running exchanges and enables clients to process data incrementally. What a stronger compliance model must still define, beyond the scope of this thesis, is the semantics of revocation: how policy changes interrupt active generated jobs, and whether already delivered chunks remain usable by the consumer after revocation.

6.5 Implemented Variants

Four paths are compared in the thesis. The first is the original DYNAMOS style and the other three are chunk-aware implementations. This distinction matters because the RabbitMQ Streams path still needs chunking before it reaches the broker. If the generated job first built one giant result and only then wrote it into RabbitMQ Streams, the client would not receive an earlier first result and the memory problem would largely remain.

Table 6.1: DYNAMOS implementation variants and the architectural boundary changed by each variant.

Variant	In-pod chain	service	Agent boundary		Buffering behaviour		Architectural purpose
Classic unary	Unary full response	gRPC,	Classic bitMQ path	RabbitMQ message	Complete result is buffered into one logical response		Baseline closest to original DYNAMOS once the gRPC size ceiling is raised.
Chunked unary	Repeated unary calls carrying row chunks		Classic bitMQ separate messages	RabbitMQ carries chunk	Chunks stay separate through aggregate, algorithm, agent, and gateway		Smallest architectural change; preserves the message-driven style.
Intra-job gRPC streaming	Long-lived <code>SendDataStream</code> through the generated service chain		Classic bitMQ separate messages	RabbitMQ carries chunk	Chunks stream inside the pod and remain chunks after leaving it		Tests native gRPC streaming where DYNAMOS already uses local gRPC.
RabbitMQ Streams at agent boundary	Chunked unary path inside the generated job		RabbitMQ stream replace queue messages for chunk transfer	records classic messages	Local chunks are written as broker stream records and reassembled downstream		Tests stream-log semantics for distributed inter-agent transfer.

The classic unary baseline uses the original DYNAMOS `main` branch at commit `a2d9531` [6]. For the benchmark, its gRPC and message-size limits are configured high enough to complete the tested row counts. This avoids treating the historical 4 MB ceiling as the result. Classic unary remains intentionally full-buffered, because that is the behaviour the streaming implementations are meant to improve. The multi-provider classic baseline also includes a minimal aggregation correction so the first provider is not appended twice during result merging. This correction makes the baseline result valid; it does not add streaming behaviour.

The three chunked paths use the streaming implementation branch at commit `923b018` [7]. Chunked unary is the least disruptive path. It keeps unary calls and the existing agent style, but it sends many smaller result messages instead of one large result. Intra-job gRPC streaming replaces repeated unary calls inside the generated service chain with one stream. That matches gRPC’s native streaming model [13], but it couples sender and receiver lifetimes more tightly. RabbitMQ Streams changes the agent boundary. It keeps the chunked local path, then uses RabbitMQ’s append-only stream abstraction for chunk transfer between agents [28].

These variants are not three independent full-stack streaming systems. They are interventions at different parts of the same DYNAMOS path. The local performance improvement of the RabbitMQ Streams variant is therefore not caused by RabbitMQ Streams alone. It is caused by the common chunk-aware data path, with RabbitMQ Streams adding a different broker representation at the agent boundary.

Table 6.2: DYNAMOS services modified on the streaming implementation branch (commit `923b018`) relative to `main` (commit `a2d9531`). The orchestrator and policy enforcer carry the selected transport, but their policy decision and job-composition responsibilities remain unchanged.

Service	Language	Role in the chunked path	Modified
<code>sql-query</code>	Python	Reads bulk results in row batches; creates chunk envelopes	Yes
<code>sql-aggregate</code>	Go	Forwards chunks without merging for bulk workloads	Yes
<code>sql-algorithm</code>	Go	Chunk-aware pass-through for bulk workloads	Yes
<code>sidecar</code>	Go	Carries chunk messages; gRPC stream endpoint (intra-job variant)	Yes
<code>agent</code>	Go	Chunk transfer at the agent boundary; RabbitMQ Streams variant	Yes
<code>api-gateway</code>	Go	Emits partial <code>providerResult</code> NDJSON events	Yes
<code>orchestrator</code>	Go	Selects and forwards the requested transport during job composition	Yes
<code>policy-enforcer</code>	Go	Propagates the transport through the existing approval response	Yes

DYNAMOS Evaluation

This chapter reports the DYNAMOS-specific benchmark used to answer RQ3. The controlled transport benchmark in Chapter 5 isolates communication mechanisms; this chapter evaluates what happens when the selected chunk-aware designs are embedded in the policy-driven DYNAMOS execution path.

7.1 Bulk Retrieval Results

Table 7.1: Average completion latency for DYNAMOS bulk retrieval. All cells completed 20/20 valid repetitions.

Implementation	50k rows	100k rows	250k rows
Classic unary	13.988s	19.343s	29.078s
Chunked unary	6.536s	6.754s	11.684s
Intra-job gRPC streaming	6.346s	8.255s	12.319s
RabbitMQ Streams boundary	5.948s	7.727s	14.066s

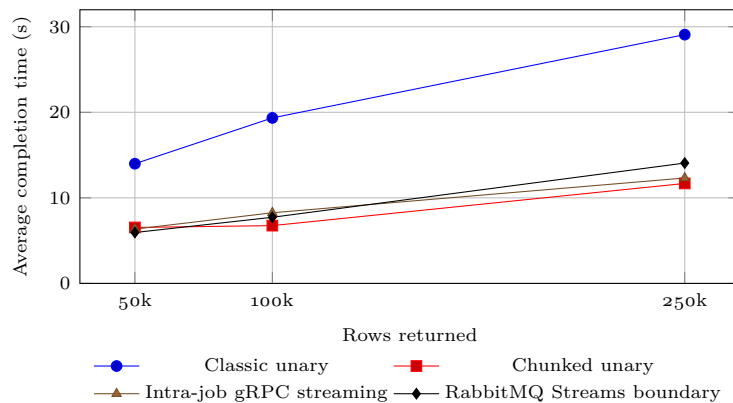


Figure 7.1: Average completion time for the local DYNAMOS bulk retrieval benchmark. Chunking is the dominant improvement: all chunk-aware variants scale better than classic unary as the returned row count grows.

Table 7.1 and Figure 7.1 show that all three chunk-aware variants reduce average completion latency compared with classic unary. At 250k rows, chunked unary is 2.49x faster than classic unary, intra-job gRPC streaming is 2.36x faster, and the RabbitMQ Streams boundary variant is 2.07x faster. The 100k row case is especially favourable to chunked unary, which finishes in 6.754s compared with 19.343s for classic unary. The relatively small increase of the chunked variants between 50k and 100k rows is explained by the fixed control-path cost analysed with Table 7.3: at these sizes much of the measured time is policy handling, orchestration, and job startup rather than transfer.

The average values include the long-tail runs that occur in the local Kubernetes deployment, so they are more conservative than medians. At 250k rows, classic unary ranged from 21.901s to 50.854s. Chunked unary ranged from 7.544s to 23.233s, intra-job gRPC streaming from 7.490s to 23.311s, and RabbitMQ Streams from 9.968s to 26.520s. This spread shows why repeated runs are needed, but the gap between classic unary and the chunked paths is large enough that it does not depend on one unusually fast or slow run.

7.2 Additional Scenario Checks

Table 7.2: Focused DYNAMOS scenario checks beyond the primary single-provider `dataThroughTtp` benchmark.

Scenario	Providers	Rows per provider	Total rows	Classic unary	Chunked unary	Intra-job gRPC streaming	RabbitMQ Streams boundary
<code>computeToData</code>	UVA,VU	50k	100k	9.622s	5.178s	4.528s	4.586s
<code>computeToData</code>	UVA,VU	250k	500k	16.620s	9.182s	7.989s	10.289s
<code>dataThroughTtp</code>	UVA,VU	50k	100k	13.750s	5.196s	4.474s	6.492s
<code>dataThroughTtp</code>	UVA,VU	250k	500k	35.396s	12.032s	11.165s	15.618s

Table 7.2 shows that the chunk-aware paths are not limited to the primary single-provider TTP case. For `computeToData`, the improvement is smaller because the request avoids part of the TTP aggregation path, but the chunked variants still complete faster than classic unary at both tested sizes. For multi-provider `dataThroughTtp`, the improvement remains large: at 250k rows per provider, the classic baseline returns 500k total rows in 35.396s, while the chunked variants complete in 11.165s to 15.618s.

These scenario checks are compatibility-oriented rather than an exhaustive benchmark matrix. Their value is that they show the implementation still works when the archetype and provider count change, and that the speedup is strongest when DYNAMOS must move large row sets through the TTP path.

7.3 Time to First Result

Read as a timeline, the averaged 100k-row chunked-unary request proceeds as follows: the benchmark client submits the HTTP request at $t = 0$; policy evaluation, orchestration, generated job creation, pod startup, and initial SQL execution together consume the first ~ 5 s; the first of 20 chunks reaches the client as a partial `providerResult` event at $t = 5.0$ s; the remaining 19 chunks drain over the next 1.8s; and the final event closes the request at $t = 6.8$ s. The classic unary baseline runs the same control path but stays silent until the single full response at $t = 19.3$ s. The comparison makes the two distinct

Table 7.3: Time-to-first-result and partial result behaviour. Classic unary has no partial results, so first result equals completion.

Implementation	Rows	First result	Done	Post-first	Partial chunks
Classic unary	50k	13.988s	13.988s	0.000s	0
Classic unary	100k	19.343s	19.343s	0.000s	0
Classic unary	250k	29.078s	29.078s	0.000s	0
Chunked unary	50k	5.778s	6.536s	0.758s	9
Chunked unary	100k	4.977s	6.754s	1.777s	19
Chunked unary	250k	5.145s	11.684s	6.539s	49
Intra-job gRPC streaming	50k	5.574s	6.346s	0.772s	9
Intra-job gRPC streaming	100k	6.250s	8.255s	2.005s	19
Intra-job gRPC streaming	250k	6.017s	12.319s	6.302s	49
RabbitMQ Streams boundary	50k	5.184s	5.948s	0.763s	9
RabbitMQ Streams boundary	100k	5.416s	7.727s	2.311s	19
RabbitMQ Streams boundary	250k	5.613s	14.066s	8.453s	49

costs visible: a fixed control-path cost that chunking does not touch, and a transfer-and-buffering cost that it removes almost entirely.

Table 7.3 shows the qualitative benefit of chunking. Classic unary cannot expose progress: the first result is the final result. The chunked variants emit partial results substantially earlier. At 250k rows, the first partial result appears after about 5.1s to 6.0s on average, while completion follows after 11.7s to 14.1s. The post-first column estimates how much of the measured time remains after the first visible chunk. It is not a pure network-transfer measurement, because the first chunk still includes policy handling, job creation, and initial SQL execution. It is nevertheless useful because it separates request startup from the observable chunk-drain phase. Read together with Table 7.1, this decomposition shows that the end-to-end ratios understate the data-path difference: every chunked variant spends roughly 5–6s before the first chunk, which is the shared policy, orchestration, and job-startup cost. End-to-end remains the headline metric since it is what a DYNAMOS client experiences, but the time-to-first-result values show that chunking also changes the interaction pattern: a client can observe progress and begin processing data before the full result has been materialised. This decomposition also matters for diagnosing whether a large request is still making progress.

7.4 Resource Usage

The resource data in Table 7.4 support the main architectural claim. At 250k rows, classic unary reaches 3,695 MiB peak memory. Chunked unary stays at 1,378 MiB, intra-job gRPC streaming at 1,495 MiB, and the RabbitMQ Streams boundary variant at 1,715 MiB. Chunking therefore reduces memory pressure across the evaluated row counts.

CPU is more nuanced. Chunked unary uses slightly less peak CPU than classic unary at 250k rows, but intra-job gRPC streaming and the RabbitMQ Streams boundary variant peak higher. This is expected. Chunking replaces one large buffer with more messages, more metadata handling, more serialisation boundaries, and, in the RabbitMQ Streams variant, broker stream handling and reassembly. The defensible claim is therefore that chunking improves latency and memory. CPU remains transport-dependent.

Table 7.4: Peak sampled resource usage during DYNAMOS bulk retrieval. CPU is peak total millicores; memory is peak total MiB across sampled Kubernetes namespaces.

Implementation	Rows	Peak CPU	Peak memory
Classic unary	50k	958m	1,571 MiB
Classic unary	100k	813m	2,046 MiB
Classic unary	250k	1,198m	3,695 MiB
Chunked unary	50k	1,059m	1,151 MiB
Chunked unary	100k	961m	1,723 MiB
Chunked unary	250k	1,097m	1,378 MiB
Intra-job gRPC streaming	50k	1,180m	1,198 MiB
Intra-job gRPC streaming	100k	1,154m	1,230 MiB
Intra-job gRPC streaming	250k	1,418m	1,495 MiB
RabbitMQ Streams boundary	50k	684m	1,161 MiB
RabbitMQ Streams boundary	100k	982m	1,436 MiB
RabbitMQ Streams boundary	250k	1,687m	1,715 MiB

7.5 Failure Modes and Robustness

During the benchmark work, several failure modes had to be controlled before the results could be interpreted reliably. Appendix A.4 summarises these cases. The important distinction is that some fixes make the baseline valid, such as correcting the multi-provider merge, while others show the architectural benefit of chunking, such as avoiding single-message size limits and full-result buffering. The RQ3 robustness claim is therefore intentionally bounded: chunking removes size-dependent failure modes, reduces full-result memory pressure, and exposes partial progress, but it does not yet prove recovery from arbitrary pod crashes, broker outages, or mid-stream policy revocation inside DYNAMOS.

Discussion

This chapter maps the results back to the research questions and discusses how far the conclusions generalise beyond the DYNAMOS case study. The emphasis is on what the measurements mean for policy-aware, dynamically composed data exchange systems, rather than on selecting a single transport in isolation.

8.1 RQ1: Transport Suitability

RQ1 asks which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems. The transport-level answer, developed in Section 5.1.8, is boundary-specific: direct gRPC mappings dominate local transfer, batched unary makes request-shaped transfer competitive by amortising per-message overhead, and the weak per-message unary result is a verdict against one-request-per-message transfer rather than against request-response communication.

Beyond the case study, this suggests that systems in this class should select transports per boundary rather than globally. Boundaries with naturally coupled endpoint lifetimes favour direct streams; request-shaped orchestration favours bounded batching; boundaries between administrative domains justify brokered streams when decoupling, retention, and replay matter more than local latency.

Brokered transports remain useful design evidence even though they are slower in the local clean matrix. RabbitMQ Streams is not validated here as a local latency improvement over chunked unary. Its value is architectural: retained records, offset-based replay, and decoupled consumption may matter more in a distributed deployment than on a single node. There is also a compliance-specific argument that the clean matrix cannot show: a retained stream is a replayable record of exactly which chunks crossed an agent boundary and in which order. In a policy-aware system, the transfer log itself can serve as an audit artifact, allowing a data steward to reconstruct what was delivered under an agreement after the fact. Direct transports provide nothing equivalent without additional logging infrastructure. NATS and Kafka are informative comparison points, but both introduce recovery, ordering, configuration, and platform shifts that make them lower-priority choices for the current DYNAMOS implementation.

8.2 RQ2: Integration with Policy-Aware Orchestration

RQ2 asks how streaming communication can be integrated while preserving dynamic service composition, policy-driven orchestration, and isolation. The DYNAMOS implementation answers this by changing the bulk data path after policy approval while leaving the control path intact. The implementation introduces an explicit chunk model at the SQL query service, preserves those chunks through aggregate and algorithm services, carries them through the agent boundary, and exposes partial result events to the client.

This design preserves the important control properties of the case-study system. Requests are still evaluated before execution, generated jobs remain the execution unit, and provider selection remains policy-driven. The streaming contribution is therefore deliberately scoped: it keeps the pre-decision enforcement model of the original system and does not add continuous, usage-control-style enforcement [27]. As Section 6.4 argues, however, the chunk model changes what enforcement can do: every chunk boundary is a potential enforcement point, revocation already works at job granularity, and an interrupted transfer leaves an auditable delivered prefix rather than an undefined state. The mechanism for chunk-boundary re-validation is in place even though the policy wiring is future work.

Two alternative integration designs were considered and rejected for this thesis. A sidecar-as-stream-proxy design, in which generated services stay unary and sidecars translate between unary calls and streams in the style of Dapr [10], would concentrate streaming complexity in infrastructure code, but it hides chunk boundaries from the services that must preserve them and would have required deeper sidecar changes than the evaluated variants. A claim-check design, in which chunks are written to shared storage and only references travel through the approved path, is the standard enterprise answer for very large payloads, but it moves the actual data outside the policy-mediated communication path and shifts the trust boundary to storage access control, which conflicts with the DYNAMOS premise that the mediated path is the enforcement surface.

The best architectural choice is again boundary-specific. Chunked unary is the strongest current local default because it gives chunking without changing the DYNAMOS execution model very much. Intra-job gRPC streaming is useful where both endpoints are already in the generated pod and a direct stream is acceptable. RabbitMQ Streams is more relevant at the inter-agent boundary, especially for future distributed deployments where retained streams, replay, and independent consumers may matter more than local latency.

8.3 RQ3: Case-Study Impact

RQ3 asks what impact the implemented streaming data paths have on scalability, performance, and robustness compared to traditional request-response transfer. The DYNAMOS-specific benchmark changes the interpretation of the isolated transport study. RQ1 showed that client-streaming gRPC is the strongest raw transport in the controlled pipeline. The DYNAMOS evaluation shows that integration fit matters as much as raw transport speed.

The central hypothesis survives validation in a more precise form: chunking

is the main source of the improvement, not the name of the transport alone. All chunk-aware DYNAMOS data paths improve local bulk-retrieval scalability compared with classic unary. They reduce completion time, expose earlier partial results, and lower peak memory usage for 50k, 100k, and 250k row result sets. The best current local default is chunked unary. Intra-job gRPC streaming is a close alternative for pod-local service chains. RabbitMQ Streams should be kept as a distributed-boundary candidate, but its advantage must be validated under cross-site conditions rather than inferred from the local latency result.

8.4 Comparison with Existing Literature

The transport-level agreement with prior benchmarks is analysed in detail in Section 5.1.7, Table 5.5. In summary, the direct-versus-brokered band structure reproduces the findings of Roohitavaf et al. [32] and Coviello et al. [4], the configuration-sensitivity of broker rankings matches Dobbelaere and Sheykh Esmaili [5] and Ibrahim et al. [17], and the single apparent divergence, Kafka’s position at the bottom of the broker band, is explained by the configuration dimensions that prior work itself identifies as decisive.

What the thesis adds to that literature is case-study-specific. First, the batched unary result shows that a request-shaped design becomes competitive when it preserves chunks and amortises per-message overhead, which matters for systems that already rely on policy-approved request orchestration. Second, the recovery scenario shows that fault behaviour separates transports that look interchangeable in a clean matrix, applying the recovery-measurement perspective of Vogel et al. [43] at the transport level. Third, the DYNAMOS evaluation shows that these transport-level findings survive integration into a policy-driven, dynamically composed execution path, which none of the prior studies covers.

8.5 Threats to Validity

8.5.1 Transport benchmark threats

Five limits are important when interpreting the transport benchmark. First, all primary RQ1 evidence comes from one local single-node Kubernetes environment. It is suitable for isolating transport overhead, but it does not capture cross-node latency, broker placement, replication, or geographically distributed deployment. Second, the benchmark uses synthetic payloads and fixed request shapes. This makes the transports comparable, but it also means that the RQ1 results should be read as mechanism-level evidence rather than as a complete data exchange workload evaluation; the in-system RQ3 measurements therefore carry more weight for the final implementation choice. Third, the tables report arithmetic means over ten repetitions. Kubernetes scheduling, broker flushing, and container resource sharing can still introduce run-to-run variation. Small differences between brokered transports should therefore not be treated as decisive. Fourth, literature comparison is directional rather than numeric: prior studies use different workloads, broker modes, deployment topologies, and persistence settings. Fifth, resource measurements are supporting evidence

rather than part of the transport ranking. They are reported only where the same sampling scope is available across compared runs, because a broker-based transport could otherwise appear more expensive simply because more of its infrastructure was included in the measurement.

8.5.2 DYNAMOS benchmark threats

The RQ3 evidence has five main limits. First, the benchmark runs on a local Kubernetes deployment, so it does not capture cross-site latency, bandwidth variation, or independent administrative domains. Second, the additional scenario checks cover two alternative request shapes, not the full space of DYNAMOS archetypes, query forms, and provider combinations. They support compatibility, but they do not replace a broader workload campaign. Third, resource usage is sampled once per second, so peak CPU and memory values can miss short spikes. Fourth, the fixed 5,000-row chunk size is justified experimentally but not optimised across all workloads. Fifth, the measured request window includes policy handling, orchestration, generated job startup, and response serialisation. That is fair for comparing user-visible DYNAMOS behaviour, but it compresses the apparent transfer-only speedup because fixed control-path costs remain present in every implementation.

Conclusion

9.1 Summary of Findings

This thesis investigated streaming communication for scalable, policy-aware data exchange systems, using DYNAMOS as a case study. The central problem was not simply that large messages are slow. The deeper issue is that policy-aware middleware must improve bulk transfer without weakening the orchestration path that decides which exchange is allowed, where it may run, and which components participate.

RQ1 asked which communication mechanisms are suitable for large-scale and continuous data exchange in microservice-based systems. The controlled benchmark shows that direct gRPC transfer is strongest for local service-to-service communication, with client-streaming giving the highest clean throughput. Batched unary is close in the clean matrix and stronger in the backpressure scenario. Brokered transports are slower locally, but they provide decoupling, retention, replay, and failure-handling options that direct streams do not provide by themselves.

RQ2 asked how streaming communication can be integrated while preserving dynamic service composition, policy-driven orchestration, and isolation. The DYNAMOS implementation keeps the existing control path intact: requests are still evaluated and composed before execution, and generated jobs remain the execution unit. The change is in the approved bulk data path. Provider-side chunks must remain chunks through aggregate, algorithm, agent, and gateway stages; otherwise the system merely hides a full-buffered result behind a streaming interface.

RQ3 validated the implemented variants on local bulk `dataThroughTtp` retrieval. All chunk-aware variants beat the classic unary baseline in average completion time and peak memory for 50k, 100k, and 250k row result sets. They also expose earlier partial results, which classic unary cannot do because its first result is its final result. Focused checks on `computeToData` and multi-provider `dataThroughTtp` show that the implementation is not confined to one request shape. The strongest local recommendation is chunked unary. It gives the best 100k and 250k averages, the lowest 250k memory peak, and the smallest architectural shift from the original message-driven design.

The overall answer is therefore boundary-specific. Streaming should be treated as a data-path change after policy approval, not as a replacement for

policy enforcement or orchestration. Direct gRPC streaming is useful where generated services have naturally coupled lifetimes. Chunked unary is the best current local default for DYNAMOS because it preserves the existing request-shaped architecture while removing full-result buffering. RabbitMQ Streams remains a distributed-boundary candidate, where retained records, replay, and independent consumers may matter more than local latency.

9.2 Contributions

This thesis makes four main contributions. First, it provides a controlled comparison of six transport mappings for large-scale and continuous microservice data exchange: client-streaming gRPC, per-message unary gRPC, batched-unary gRPC, RabbitMQ Streams, NATS JetStream, and Kafka. Second, it identifies chunk preservation as the central architectural requirement for improving large-result transfer in policy-aware generated workflows. Third, it implements and evaluates three chunk-aware DYNAMOS data paths: chunked unary transfer, intra-job gRPC streaming, and RabbitMQ Streams at the agent boundary. Fourth, it provides a benchmark corpus and appendices that document the relevant run scopes, parameter rationale, environment, provenance, failure modes, clean-matrix results, and resource measurements.

9.3 Limitations

The conclusions are strongest for the local Kubernetes environments and workloads evaluated in this thesis. The RQ₁ benchmark isolates transport overhead on a single node, while the RQ₃ benchmark validates DYNAMOS bulk retrieval locally. Cross-site latency, broker placement, replication, independent administrative domains, broader DYNAMOS archetypes, and mid-stream policy changes remain outside the validated scope. Resource usage is sampled rather than continuously traced, and the selected 5,000-row chunk size is an experimental configuration rather than a global optimum. These limitations do not invalidate the observed local improvements, but they bound the generality of the recommendations.

9.4 Future Work

The most important future work is distributed validation. The local DYNAMOS benchmark proves that the basic chunk-aware path works before it is meaningful to test it across sites. A natural next step is Scattered Directive, which extends DYNAMOS with cross-node microservice chains for policy-aware VFL workflows and uses FABRIC to automate distributed deployment [8, 19].

In that setting, bulk transfer can become part of a larger VFL-style workflow. Parties may hold different features for overlapping entities, and a TTP or shared execution component may coordinate aligned intermediate values, labels, embeddings, model updates, or other training artefacts. The current thesis does not claim to validate a complete VFL training workload. It validates the communication path that such a workload would depend on when large intermediate results have to cross generated services and agent boundaries.

FABRIC provides programmable compute and storage resources connected by high-speed dedicated links, which makes it a suitable environment for testing policy-aware data exchange when the TTP and data providers are separated across sites [2, 9]. A later FABRIC experiment should keep the RQ3 metrics from this section and add network throughput, broker lag or stream backlog, cross-site tail latency, failure recovery, concurrent-job scaling, and policy-change responsiveness. RabbitMQ Streams deserves special attention in that setting because its local overhead may be outweighed by retained-stream and replay semantics once agents are no longer co-located.

The second future-work direction is stronger policy handling during active streams. The evaluated implementation preserves pre-stream policy approval, but it does not solve mid-stream revocation or continuous policy enforcement. The usage-control literature provides the target model: ongoing authorisation with revocation during use [15, 27]. A concrete next step is chunk-boundary re-validation as outlined in Section 6.4, followed by lease- or capability-based transfer rights in which the policy engine issues a time- or volume-bounded transfer capability that sidecars enforce and that expires unless renewed. A complete policy-aware streaming middleware should further define how policy changes interrupt active generated jobs, how already delivered chunks are audited, and how downstream services should react when a long-running approved exchange becomes invalid. A third, smaller direction is backpressure-aware orchestration: stream lag and backlog signals from the data path could inform orchestration decisions such as admission and placement, connecting the streaming layer back to the adaptive part of the middleware.

Benchmark and Implementation Details

A.1 Benchmark Preset Definitions

This appendix documents the complete preset definitions used in the benchmark evidence reported in the thesis. Each preset specifies the workload family, payload configuration, concurrency level, target rate where applicable, failure injection parameters, and resource overlay tier. The four reported presets are: *synthetic-clean* for bulk transfer, *synthetic-continuous* for steady-rate pacing, *synthetic-backpressure* for slow-consumer overload, and *synthetic-recovery* for forced-restart resilience. The harness also contains a CSV replay mode, but those results are not used in the current thesis argument.

Table A.1: Benchmark presets used in the current transport study.

Preset	Key parameters	Purpose
synthetic-clean	Synthetic payloads from 256 B to 16 KiB; concurrency 1–16; ten repetitions; no injected failure	Baseline bulk-transfer throughput, latency, and correctness matrix
synthetic-continuous	1 KiB payloads, 1,000 msg/s target, concurrency 4 and 8, ten repetitions	Steady-rate pacing and jitter comparison
synthetic-backpressure	1 KiB payloads, 4,000 msg/s target, concurrency 8, 2 ms sink delay, ten repetitions	Slow-consumer overload and queueing behaviour
synthetic-recovery	4 KiB payloads, 2,000 msg/s target, concurrency 8, transformer restart after 3 s, 120 retries at 250 ms back-off, ten repetitions	Restart recovery, replay, and continuity analysis

A.2 Full Run Matrix

This appendix summarises the canonical run scopes used in the current thesis. The body of the thesis presents the most decision-relevant subsets, while the benchmark result directory stores the corresponding per-run JSON, NDJSON, and CSV exports generated by the execution scripts. The chapter is based on the single-node six-transport `synthetic-clean` matrix and the matching single-node scenario suite, each with ten repetitions. Clustered broker runs are kept as supporting variability context rather than as the primary thesis evidence.

Table A.2: Execution scope of the current benchmark corpus.

Scope	Transports	Variable dimensions	Reps	Role in thesis
Single-node clean matrix	Client-streaming, batched unary, RabbitMQ, NATS, Kafka	Payload 256 B–16 KiB, concurrency 1–16	10	Full bulk-transfer baseline (Chapter 5)
Single-node continuous	All six	Concurrency 4 and 8, 1 KiB, 1,000 msg/s	10	Pacing and jitter evidence
Single-node backpressure	All six	1 KiB, concurrency 8, 4,000 msg/s, 2 ms sink delay	10	Overload and queueing evidence
Single-node recovery	All six	4 KiB, concurrency 8, transformer re-start	10	Recovery and correctness evidence
Clustered clean (3-broker)	RabbitMQ, NATS, Kafka	4 KiB, concurrency 8	3	Supporting broker variability context
Clustered continuous (3-broker)	RabbitMQ, NATS, Kafka	1 KiB, concurrency 8, 1,000 msg/s	3	Clustered pacing under broker replication
Clustered backpressure (3-broker)	RabbitMQ, NATS, Kafka	1 KiB, concurrency 8, 4,000 msg/s, 2 ms delay	3	Clustered overload behaviour
Clustered recovery (3-broker)	RabbitMQ, NATS, Kafka	4 KiB, concurrency 8, transformer re-start	3	Clustered recovery correctness and variability

A.3 Run Provenance

This appendix records where the source code, runner scripts, and result files for the thesis evidence are located. The public thesis and benchmark repository is cited as the experiment artifact [36]. DYNAMOS itself is evaluated through two pinned code states: the classic baseline and the streaming implementation branch.

Table A.3: Repository, script, and result locations used for thesis reproducibility.

Artifact	Location or identifier
Thesis and synthetic benchmark repository	https://github.com/neburpoots/Streaming_Techniques_Experiment
DYNAMOS classic baseline	a2d9531 on the DYNAMOS main branch [6]
DYNAMOS streaming implementation	923b018 on feature/streaming-implementations [7]
RQ1 benchmark runner	experiments/grpc-streaming-baseline/scripts/run-k8s-matrix.sh
RQ1 Kubernetes manifests	experiments/grpc-streaming-baseline/k8s/
RQ1 result directory	experiments/grpc-streaming-baseline/results/
Main RQ1 NATS rerun prefix	rq1-nats-fetchwait-20260621-
DYNAMOS benchmark scripts and raw outputs	Stored alongside the pinned DYNAMOS experiment branches and summarised in the thesis result tables
Broker images	rabbitmq:3-management, nats:2.10-alpine, apache/kafka:3.9.1

A.4 DYNAMOS Failure-Mode Notes

Table A.4: Failure modes observed or accounted for during the DYNAMOS benchmark work.

Failure mode	Affected path	Observation	Treatment	Robustness claim
Default gRPC message ceiling	Original classic unary	Original DYNAMOS documentation reports a practical limit around 4 MB, or roughly 30k rows in that setup	Increased benchmark message limits for the baseline so the comparison measures buffering cost rather than an artificial ceiling	Chunking removes the dependency on one very large response message
Full-result buffering	Classic unary and early bulk prototypes	Large results were materialised repeatedly before the client saw progress	Preserved 5k-row chunks through SQL, aggregate, algorithm, agent, and gateway stages	Supported by lower memory peaks and earlier partial results
Stale generated jobs	Repeated classic unary runs	Later repetitions could return fast 502 or empty-provider responses when old generated jobs remained	Cleaned generated surf, uva, and vu jobs outside the measured request window	Not a streaming advantage; handled for benchmark fairness
Multi-provider merge error	Classic multi-provider baseline	The first provider could be appended twice during aggregation	Applied a minimal aggregation correction before reporting multi-provider baseline timings	Baseline correctness fix, not a streaming feature
Timeout and empty-provider reporting	Early large-result runs	Some failures appeared as missing final results or empty provider bodies	Made timeout/error outcomes visible in benchmark output and separated clean errors from valid completions	Improves diagnosability; not a full crash-recovery proof

A.5 Detailed Clean-Matrix Results

This appendix records the completed six-transport `synthetic-clean` matrix at the payload and concurrency level. Values are arithmetic means over ten repetitions. The result directory stores the per-run JSON and CSV files for the individual repetitions.

Table A.5: Synthetic bulk-transfer throughput by message size, averaged over ten repetitions and concurrency levels 1, 4, 8, and 16.

Msg size	Stream msg/s	Batched unary	Unary msg/s	RabbitMQ msg/s	NATS msg/s	Kafka msg/s
256 B	101,778	87,393	3,555	22,553	19,105	17,486
1 KiB	68,038	61,882	3,323	17,867	16,667	14,485
4 KiB	29,928	35,167	3,066	10,077	12,178	9,691
16 KiB	12,913	11,943	2,322	4,095	5,394	3,513

Table A.6: Detailed synthetic-clean message throughput (msg/s) by payload size and concurrency, averaged over ten repetitions.

Msg size	Concur.	Stream	Batched	Unary	RabbitMQ	NATS	Kafka
256 B	1	102,181	50,874	933	19,183	16,617	20,146
	4	100,739	86,526	2,844	25,848	20,470	16,876
	8	114,765	101,825	4,191	24,362	18,884	17,270
	16	89,428	110,346	6,252	20,818	20,449	15,651
1 KiB	1	70,938	34,060	966	16,008	14,211	16,882
	4	73,665	70,253	2,542	18,030	16,787	13,889
	8	65,887	73,116	4,150	19,021	17,689	15,332
	16	61,663	70,098	5,632	18,410	17,979	11,837
4 KiB	1	29,454	22,985	988	8,737	10,003	9,760
	4	31,524	36,309	2,507	10,663	12,561	10,416
	8	30,307	41,008	3,773	10,285	13,286	10,031
	16	28,426	40,364	4,995	10,622	12,863	8,558
16 KiB	1	10,287	9,335	854	3,027	4,819	3,331
	4	14,022	12,631	2,097	4,691	5,525	3,527
	8	13,836	13,371	2,580	4,530	5,997	3,746
	16	13,507	12,435	3,759	4,132	5,236	3,450

Table A.7: Detailed synthetic-clean data throughput (MiB/s) by payload size and concurrency, averaged over ten repetitions.

Msg size	Concur.	Stream	Batched	Unary	RabbitMQ	NATS	Kafka
256 B	1	24.95	12.42	0.23	4.68	4.06	4.92
	4	24.59	21.12	0.69	6.31	5.00	4.12
	8	28.02	24.86	1.02	5.95	4.61	4.22
	16	21.83	26.94	1.53	5.08	4.99	3.82
1 KiB	1	69.28	33.26	0.94	15.63	13.88	16.49
	4	71.94	68.61	2.48	17.61	16.39	13.56
	8	64.34	71.40	4.05	18.58	17.27	14.97
	16	60.22	68.46	5.50	17.98	17.56	11.56
4 KiB	1	115.05	89.79	3.86	34.13	39.07	38.12
	4	123.14	141.83	9.79	41.65	49.07	40.69
	8	118.39	160.19	14.74	40.18	51.90	39.19
	16	111.04	157.67	19.51	41.49	50.25	33.43
16 KiB	1	160.74	145.86	13.34	47.30	75.30	52.05
	4	219.09	197.37	32.76	73.30	86.33	55.11
	8	216.18	208.92	40.32	70.78	93.70	58.53
	16	211.05	194.30	58.73	64.56	81.82	53.90

Table A.8: Detailed synthetic-clean p_{95} latency (ms) by payload size and concurrency, averaged over ten repetitions.

Msg size	Concur.	Stream	Batched	Unary	RabbitMQ	NATS	Kafka
256 B	1	17.3	1.3	0.7	249.2	648.2	176.7
	4	61.7	6.9	1.0	320.1	561.2	543.0
	8	81.1	29.1	1.5	336.1	674.5	562.6
	16	87.6	40.4	2.1	404.6	664.0	934.3
1 KiB	1	37.8	3.0	0.5	393.7	740.4	191.4
	4	92.7	12.5	1.2	444.4	719.6	642.8
	8	94.1	43.0	1.5	455.4	710.3	714.4
	16	110.5	59.3	1.9	453.4	680.9	1,364.3
4 KiB	1	40.1	7.2	0.7	580.8	997.5	140.2
	4	107.1	41.5	0.9	696.4	897.3	867.9
	8	140.9	60.3	1.2	965.2	848.8	1,099.4
	16	193.1	79.8	2.4	795.3	985.5	1,681.5
16 KiB	1	36.8	35.8	0.6	1,977.9	2,015.6	95.5
	4	106.5	68.3	1.6	1,540.8	2,076.9	1,984.0
	8	259.0	93.7	3.0	1,958.8	2,048.9	2,015.2
	16	254.6	248.5	4.7	2,396.9	2,577.1	5,977.7

A.6 Resource Utilisation Details

This appendix summarises supporting resource metrics for diagnostic Kubernetes scenario runs where CPU and memory data are most informative. For brokered modes, the broker is reported as a separate role because its buffering layer is a core part of the transport cost. These tables are not used for the primary RQ1 ranking, which is based on the completed ten-repetition throughput, latency, and correctness corpus. The six-transport scenario resource exports remain in the benchmark result directory; the tables below are retained as role-level context where the sampling scope is comparable.

Table A.9: Continuous scenario CPU usage by role (avg./peak %).

Mode	Concur.	Producer	Transformer	Sink	Broker
Client-streaming	4	23.40 / 23.40	8.99 / 24.20	5.61 / 14.90	–
Client-streaming	8	–	9.31 / 25.60	5.64 / 15.50	–
Unary	4	35.70 / 35.70	17.88 / 39.10	11.12 / 24.10	–
Unary	8	–	15.54 / 33.70	9.72 / 20.90	–
RabbitMQ Streams	4	7.92 / 8.30	9.87 / 15.60	6.91 / 10.90	29.88 / 43.30
RabbitMQ Streams	8	7.88 / 8.20	10.46 / 15.80	7.50 / 11.70	33.00 / 46.50
NATS JetStream	4	–	12.62 / 24.90	5.60 / 15.40	18.37 / 40.90
NATS JetStream	8	–	16.23 / 23.70	6.45 / 14.20	22.74 / 38.50
Kafka	4	–	4.68 / 5.70	0.69 / 1.90	51.52 / 52.90
Kafka	8	–	8.58 / 8.80	1.00 / 2.20	41.13 / 58.80

Table A.10: Recovery scenario CPU usage by role (avg./peak %).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	31.60 / 31.60	12.64 / 31.60	3.89 / 17.50	–
Unary	4.60 / 6.00	2.45 / 9.10	2.56 / 6.10	–
RabbitMQ Streams	8.84 / 10.20	13.38 / 17.10	7.51 / 13.90	31.76 / 49.90
NATS JetStream	0.30 / 0.30	17.29 / 25.20	7.28 / 16.20	20.99 / 43.50
Kafka	0.40 / 0.40	7.63 / 8.10	1.44 / 2.50	46.77 / 75.40

Table A.11: Recovery scenario memory usage by role (avg./peak MiB).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	7.00 / 7.00	3.40 / 7.00	2.33 / 7.00	–
Unary	7.00 / 7.00	3.08 / 7.00	4.53 / 7.00	–
RabbitMQ Streams	10.00 / 10.00	25.72 / 31.00	5.78 / 8.00	490.91 / 511.00
NATS JetStream	8.00 / 8.00	14.00 / 14.00	9.75 / 15.00	306.06 / 537.00
Kafka	10.00 / 10.00	8.00 / 8.00	6.53 / 9.00	449.71 / 478.00

References

- [1] Apache Software Foundation. *Apache Kafka*. 2026. url: <https://kafka.apache.org/> (visited on 03/04/2026).
- [2] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, K.-C. Wang, T. Lehman, P. Ruth and E. Deelman. “FABRIC: A National-Scale Programmable Experimental Network Infrastructure”. In: *IEEE Internet Computing* 23.6 (2019), pp. 38–47. doi: 10.1109/MIC.2019.2958545.
- [3] B. Çiftçi and B. Çiloğlugil. “A Review of Comparative Studies on Performance Evaluation of Communication Mechanisms for Microservices”. In: *Computational Science and Its Applications – ICCSA 2025*. Vol. 15650. Lecture Notes in Computer Science. Springer Nature Switzerland AG, 2025, pp. 98–113. doi: 10.1007/978-3-031-96962-1_7.
- [4] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone and S. Chakradhar. “DataXc: Flexible and efficient communication in microservices-based stream analytics pipelines”. In: *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCCom / CyberSciTech*. 2022. doi: 10.1109/DASC/PiCom/CBDCCom/Cy55231.2022.9927837.
- [5] P. Dobbelaere and K. Sheykh Esmaili. *Kafka versus RabbitMQ*. arXiv preprint. 2017. doi: 10.48550/arXiv.1709.00333. arXiv: 1709.00333 [cs.DC].
- [6] DYNAMOS Contributors. *DYNAMOS source repository, main branch, commit a2d9531*. 2026. url: <https://github.com/neburpoots/DYNAMOS/tree/a2d953166c369260e6014f41bd988ee3ae45592d> (visited on 09/06/2026).
- [7] DYNAMOS Contributors. *DYNAMOS streaming implementations prototype, feature/streaming-implementations branch, commit 923b018*. 2026. url: <https://github.com/neburpoots/DYNAMOS/tree/923b018e622e30ed6141a3e9c925417ce9150e5d> (visited on 25/05/2026).
- [8] DYNAMOS-UVA. *Scattered Directive source repository*. 2026. url: <https://github.com/DYNAMOS-UVA/Scattered-Directive> (visited on 25/05/2026).
- [9] FABRIC Testbed. *About FABRIC*. 2026. url: <https://portal.fabric-testbed.net/about/about-fabric> (visited on 25/05/2026).

- [10] J. Figueira and C. Coutinho. “Developing Self-Adaptive Microservices”. In: *Procedia Computer Science* 232 (2024), pp. 264–273. doi: 10.1016/j.procs.2024.01.026.
- [11] M. Fragkoulis, P. Carbone, V. Kalavri and A. Katsifodimos. “A Survey on the Evolution of Stream Processing Systems”. In: *VLDB Journal* 33.2 (2024), pp. 507–541. doi: 10.1007/s00778-023-00819-8.
- [12] Y. Gan and C. Delimitrou. “The Architectural Implications of Cloud Microservices”. In: *IEEE Computer Architecture Letters* 17.2 (2018), pp. 155–158. doi: 10.1109/LCA.2018.2839189.
- [13] gRPC Authors. *Core Concepts, Architecture and Lifecycle*. Last modified 2024-11-12. 2024. url: <https://grpc.io/docs/what-is-grpc/core-concepts/> (visited on 03/04/2026).
- [14] B. Guntupalli and S. Vamshi Ch. “Designing Microservices That Handle High-Volume Data Loads”. In: *International Journal of AI, Big Data, Computational and Management Studies* 4.4 (2023), pp. 76–87. doi: 10.63282/3050-9416.IJAIBDCMS-V4I4P109.
- [15] A. Hariri, A. Ibrahim, B. Alangot, S. Bandopadhyay, A. La Marra, A. Rosetti, H. Joumaa and T. Dimitrakos. “UCON+: Comprehensive Model, Architecture and Implementation for Usage Control and Continuous Authorization”. In: *Collaborative Approaches for Cyber Security in Cyber-Physical Systems*. Ed. by T. Dimitrakos, J. Lopez and F. Martinelli. Advanced Sciences and Technologies for Security Applications. Springer, 2023, pp. 209–226. doi: 10.1007/978-3-031-16088-2_10.
- [16] S. Henning and W. Hasselbring. “How to Measure Scalability of Distributed Stream Processing Engines?” In: *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE ’21)*. ACM, 2021, pp. 85–88. doi: 10.1145/3447545.3451190.
- [17] A. G. Ibrahim, R. P. Lopes, J. Rufino and P. Leitão. “On the Impact of Message Brokers Implementations in the Choreography of Microservices”. In: *Open Lecture Series 2A (OL2A 2025)*. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026. doi: 10.1007/978-3-032-00137-5_1.
- [18] International Data Spaces Association. *IDS Usage Control Policies*. International Data Spaces Association. 2023. url: <https://international-data-spaces-association.github.io/DataspaceConnector/Documentation/v6/UsageControl> (visited on 06/07/2026).
- [19] J. Jongejans, A. Koufakis and A. Oprescu. “Vertical Federated Learning on Scattered Directive: Enforcing Policies on VFL Workflows”. In: *Proceedings of the 2025 IEEE International Conference on Big Data (BigData)*. 2025, pp. 267–274. doi: 10.1109/BIGDATA66926.2025.11401076.
- [20] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen and V. Markl. “Benchmarking Distributed Stream Data Processing Systems”. In: *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE. 2018, pp. 1519–1530. doi: 10.1109/ICDE.2018.00169.

- [21] H. Kraft and R. Johansson. “Evaluating RPC for Cloud-Native 5G Mobile Network Applications”. Master’s thesis. Chalmers University of Technology and University of Gothenburg, 2020. url: <https://hdl.handle.net/20.500.12380/302298>.
- [22] V. Lähtevänoja. “Communication Methods and Protocols Between Microservices on a Public Cloud Platform”. MA thesis. Espoo, Finland: Aalto University, School of Science, May 2021. url: <https://urn.fi/URN:NBN:fi:aalto-202106207501>.
- [23] S. Lote, P. K B and D. Prater. “Real-time data stream processing in large-scale systems”. In: *World Journal of Advanced Research and Reviews* 15.3 (2022), pp. 560–570. doi: 10.30574/wjarr.2022.15.3.0903.
- [24] O.-C. Marcu and P. Bouvry. *Colocating Real-time Storage and Processing: An Analysis of Pull-based versus Push-based Streaming*. arXiv preprint. 2022. doi: 10.48550/arXiv.2211.05857. arXiv: 2211.05857 [cs.DC].
- [25] S. Neumaier, G. Havur and T. Pellegrini. “Towards an Architecture for Policy-Aware Decentral Dataset Exchange”. In: *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. url: https://www.researchgate.net/publication/358816711_Towards_an_Architecture_for_Policy-Aware_Decentral_Dataset_Exchange.
- [26] J. Nuikka. “Comparison of Cloud Native Messaging Technologies”. MA thesis. Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. url: <https://urn.fi/URN:NBN:fi:tuni-202104223314>.
- [27] J. Park and R. Sandhu. “The UCON_{ABC} Usage Control Model”. In: *ACM Transactions on Information and System Security* 7.1 (2004), pp. 128–174. doi: 10.1145/984334.984339.
- [28] RabbitMQ Team. *Streams and Superstreams (Partitioned Streams)*. 2026. url: <https://www.rabbitmq.com/docs/streams> (visited on 03/04/2026).
- [29] J. A. Rasheedh and S. Saradha. “Reactive Microservices Architecture Using a Framework of Fault Tolerance Mechanisms”. In: *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021. doi: 10.1109/ICESC5142.2.2021.9532893.
- [30] S. Ris, J. Araujo and D. Beserra. “A Systemic Mapping of Methods and Tools for Performance Analysis of Data Streaming with Containerized Microservices Architecture”. In: *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*. 2023, pp. 1–6. doi: 10.23919/CISTI58278.2023.10211834.
- [31] Ritu, S. Arora, A. Bhardwaj, A. Kukkar and S. Kaur. “A Comparative Analysis of Communication Efficiency: REST vs. gRPC in Microservice-Based Ecosystems”. In: *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024. doi: 10.1109/INNOCOMP63224.2024.00107.

- [32] M. Roohitavaf, K. Ren, G. Zhang and S. Ben-Romdhane. *LogPlayer: Fault-tolerant Exactly-once Delivery using gRPC Asynchronous Streaming*. arXiv preprint. 2019. doi: 10.48550/arXiv.1911.11286. arXiv: 1911.11286 [cs.DC].
- [33] T. Sharvari and K. Sowmya Nag. *A Study on Modern Messaging Systems: Kafka, RabbitMQ and NATS Streaming*. arXiv preprint. 2019. doi: 10.48550/arXiv.1912.03715. arXiv: 1912.03715 [cs.DC].
- [34] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar and S. Kalambur. “Benchmarking gRPC Protocol on Various Virtualization Technologies”. In: *ICT4SD 2025*. Vol. 1654. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, pp. 316–329. doi: 10.1007/978-3-032-06697-8_32.
- [35] C. Starck and J. Ghofrani. “Improving Load Balancing of Long-lived Streaming RPCs for gRPC-enabled Inter-service Communication”. In: *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*. Ed. by S. Böhm and D. Lübke. Vol. 3386. CEUR Workshop Proceedings. Hannover, Germany, 2023, pp. 45–51. url: <https://ceur-ws.org/Vol-3386/>.
- [36] R. Stoop. *Streaming Techniques Experiment: Synthetic Transport Benchmark and Thesis Artifacts*. 2026. url: https://github.com/neburpoots/Streaming_Techniques_Experiment (visited on 22/06/2026).
- [37] J. Stutterheim. “DYNAMOS: Dynamically Adaptive Microservice-based OS: A Middleware for Data Exchange Systems”. MA thesis. University of Amsterdam, 2023.
- [38] J. Stutterheim, A. Mifsud and A. Oprescu. “DYNAMOS: Dynamic Microservice Composition for Data-Exchange Systems, Lessons Learned”. In: *Proceedings of the 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2024, pp. 8–15. doi: 10.1109/ICSA-C63560.2024.00008.
- [39] The NATS Authors. *JetStream*. 2026. url: <https://docs.nats.io/nats-concepts/jetstream> (visited on 03/04/2026).
- [40] The NATS Authors. *Overview*. 2026. url: <https://docs.nats.io/nats-concepts/overview> (visited on 03/04/2026).
- [41] The NATS Authors. *Request-Reply*. 2026. url: <https://docs.nats.io/nats-concepts/core-nats/reqreply> (visited on 03/04/2026).
- [42] G. Thiagarajan, V. Bist and P. Nayak. “Strengthening gRPC Security in Microservices: A Proxy-based Approach for mTLS, JWT, and RBAC Enforcement”. In: *International Journal of Computer Applications* 187.28 (Aug. 2025). doi: 10.5120/ijca2025925359.
- [43] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl and R. Rabiser. “A Comprehensive Benchmarking Analysis of Fault Recovery in Stream Processing Frameworks”. In: *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems (DEBS ’24)*. ACM, 2024, pp. 171–182. isbn: 979-8-4007-0443-7. doi: 10.1145/3629104.3666040.

Acronyms

Acronym	Meaning
API	Application Programming Interface
CCI	Complex Cyber-Infrastructure
CPU	Central Processing Unit
DYNAMOS	Dynamically Adaptive Microservice-based Operating System
GC	Garbage Collection
gRPC	gRPC Remote Procedure Calls
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
IPC	Inter-Process Communication
ISO	International Organization for Standardization
JVM	Java Virtual Machine
JSON	JavaScript Object Notation
NDJSON	Newline Delimited JSON
REST	Representational State Transfer
RPC	Remote Procedure Call
RQ	Research Question
SOTA	State of the Art
TTP	Trusted Third Party
UvA	University of Amsterdam
VFL	Vertical Federated Learning

